Machine Learning with Applications 25 (2026) 100984

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

A leakage-free benchmark of offline policy learning for SOC alert triage

Emmanuel Okafor *

Center of Artificial Intelligence for Security, Naif Arab University for Security Sciences, Riyadh, 11452, Saudi Arabia
Department of Cybersecurity and Digital Forensics, Naif Arab University for Security Sciences, Riyadh, 11452, Saudi Arabia

ARTICLE INFO

Keywords:
Offline reinforcement learning
Security operations center
Alert triage
Temporal evaluation
Intrusion detection
Decision transformer

ABSTRACT

Security operations centers (SOCs) must triage alert volumes that far exceed analyst capacity, a sequential, cost-sensitive decision problem well suited to reinforcement learning (RL). Yet there are two obstacles that keep prior work in controlled settings: online exploration is unsafe on live infrastructure, and public intrusion datasets expose only independent flow records, not the decision trajectories offline RL requires; evaluations further suffer from temporal leakage and mean-only reporting that make degenerate policies look deployable. We address these limitations with three components: a pseudo-trajectory reconstruction that converts flat flow tables into offline decision episodes by causally grouping flows on host identifiers, segmenting them temporally, and annotating transitions with a multi-objective reward; a strictly leakage-free temporal protocol (chronological partitioning, training-only normalization, expanding-window cross-validation, and a post-split audit); and multi-metric, stability-aware reporting. We benchmark seven offline policy-learning agents spanning four families — memoryless and recurrent imitation (Behavioral Cloning), advantage-weighted recurrent policies (GRU, LSTM), value-based offline RL (CQL, IQL), and a return-conditioned sequence model (Decision Transformer) — across five seeds with Friedman, Nemenyi, and Wilcoxon tests on two identifier-complete datasets, UNSW-NB15 and CIC-IDS-2017. The reward-free recurrent imitator (Recurrent Behavioral Cloning) is the strongest and most generalizable agent on both datasets (Matthews correlation coefficient, MCC, 0.752 on UNSW-NB15 and 0.621 on CIC-IDS-2017), improving detection MCC over the memoryless imitation baseline by about 59% and 26% respectively, and exceeding the advantage-weighted recurrent policy by 0.086 and 0.095—evidence that the gains come from recurrence and trajectory context, not reward engineering. The value-based methods and the Decision Transformer degenerate into trivial detectors that a collapse in F1 and ROC-AUC conceals but MCC reveals. We deploy the framework in an advisory, human-in-the-loop mode with a deterministic severity floor that prevents any medium- or high-severity alert from being auto-dismissed, and with containment analyst-owned.

1. Introduction

Security operations centers (SOCs) are increasingly overwhelmed by the scale, velocity, and heterogeneity of modern network telemetry. Contemporary intrusion detection systems (IDSs) generate massive streams of alerts, yet only a small proportion correspond to genuine malicious activity. Analysts must therefore triage alerts under severe time constraints, determining whether an event should be dismissed, investigated further, or escalated for containment. This process is inherently sequential and cost-sensitive: dismissing a malicious alert may permit lateral movement or compromise escalation, whereas unnecessarily containing benign traffic can disrupt operational continuity and generate substantial organizational cost. Although machine learning has been widely applied to intrusion detection, most existing approaches frame the problem as static classification, anomaly scoring, or alert prioritization (Okafor & Lee, 2026) rather than sequential operational decision-making (Huiying, 2026; Mukil et al., 2026; Neto et al., 2025). Consequently, these formulations may not fully account for the downstream consequences and temporal dependencies inherent in real SOC workflows.

Reinforcement learning (RL) provides a principled framework for modeling such sequential decision processes by optimizing policies against long-term operational reward. Recent studies demonstrate that RL and deep reinforcement learning (DRL) methods can substantially improve adaptability and resilience in intrusion detection systems, particularly in dynamic environments such as IoT, cloud, and software-defined networks (Louati et al., 2024; Mpoporo et al., 2026; Yang et al., 2026). However, the overwhelming majority of prior RL-for-IDS research relies on online interaction with simulated environments

* Correspondence to: Center of Artificial Intelligence for Security, Naif Arab University for Security Sciences, Riyadh, 11452, Saudi Arabia.
E-mail address: eokafor@nauss.edu.sa.

https://doi.org/10.1016/j.mlwa.2026.100984
Received 6 July 2026; Received in revised form 12 August 2026; Accepted 21 August 2026
Available online 29 August 2026
2666-8270/© 2026 The Author. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

or synthetically reconstructed Markov decision processes, allowing unrestricted exploratory actions during training (Gueriani et al., 2023; Jamshidi et al., 2025). Such assumptions are incompatible with operational cybersecurity settings, where exploratory containment actions against live infrastructure are unacceptable. Offline reinforcement learning, which learns entirely from previously logged interactions without further environment exploration, therefore represents a more realistic and deployment-aligned paradigm for SOC automation (Levine et al., 2020). Nonetheless, applying offline RL to cybersecurity introduces significant challenges related to distributional shift, sparse and imbalanced security outcomes, and the absence of realistic sequential decision data.

Recent advances in offline RL have produced two dominant methodological paradigms. Conservative value-based methods, including Conservative Q-Learning (CQL) (Kumar et al., 2020) and Implicit Q-Learning (IQL) (Kostrikov et al., 2021), mitigate overestimation of out-of-distribution actions by constraining learned policies toward the support of the logged dataset. In contrast, sequence-modeling approaches such as the Decision Transformer (DT) (Chen et al., 2021) reinterpret reinforcement learning as return-conditioned sequence prediction, leveraging Transformer architectures to generate actions autoregressively without explicit value estimation. These paradigms embody distinct operational trade-offs in SOC environments. Conservative value-based agents may depart from historical analyst behavior when reward optimization justifies more aggressive intervention, whereas imitation-oriented and sequence models more faithfully reproduce the inherently cautious behavior embedded within analyst demonstrations. Understanding how these differing inductive biases influence cybersecurity triage decisions remains largely unexplored, particularly under realistic offline constraints and temporally consistent evaluation protocols.

A further limitation in current cybersecurity ML and RL research concerns evaluation integrity. Numerous studies have shown that data leakage and temporally inconsistent train–test protocols can substantially inflate reported performance in intrusion detection tasks (Arp et al., 2022) and data-driven applied sciences (Kapoor & Narayanan, 2023). Temporal leakage is especially problematic in network security because future traffic patterns may inadvertently influence model training through random dataset splitting or globally fitted preprocessing pipelines. Despite extensive warnings regarding these methodological flaws, many RL-based IDS studies continue to evaluate under unrealistic assumptions that fail to reflect deployment conditions (Jamshidi et al., 2025). At the same time, widely used intrusion datasets such as UNSW-NB15 (Moustafa & Slay, 2015) provide flat collections of independent flow records rather than the sequential trajectories required for offline RL. As a result, existing work seldom addresses how to reconstruct operational decision processes from static intrusion logs while simultaneously ensuring leakage-free temporal evaluation.

This study addresses these gaps through two methodological contributions: pseudo-trajectory reconstruction and leakage-free temporal evaluation for offline policy learning in SOC alert triage. We first introduce a reconstruction framework that converts flat flow-based intrusion records into temporally ordered analyst decision trajectories through causal flow grouping and multi-objective reward assignment. We then implement a strictly chronological evaluation protocol in which partitioning, normalization, and preprocessing are performed exclusively using training-period statistics, followed by a post-split audit to verify the absence of temporal leakage. Using this framework, we benchmark seven offline policy-learning agents spanning imitation baselines (memoryless and recurrent Behavioral Cloning, BC), reward-weighted recurrent sequence policies (GRU, LSTM), value-based offline RL (CQL, IQL), and a return-conditioned sequence model (Decision Transformer) under the analyst triage actions Dismiss, Investigate, and Contain. To mitigate severe imbalance in containment actions, we employ class-weighted focal loss and evaluate all methods across multiple random seeds using Friedman and Nemenyi statistical tests. Our results show that the recurrent, advantage-weighted sequence policies achieve the most stable recovery of analyst triage behavior across seeds, whereas the value-based agents and the Decision Transformer degenerate into trivial detectors, highlighting stability as a critical evaluation criterion for offline learning in cybersecurity. The visualization of our proposed system is presented in Fig. 1. In summary, this paper makes the following contributions:

- a *pseudo-trajectory reconstruction* procedure that converts flow-based intrusion datasets into offline decision trajectories through causal host grouping, temporal segmentation, and multi-objective reward assignment;
- a leakage-free temporal evaluation protocol — chronological partitioning, train-only normalization, and explicit post-split leakage auditing — extended to expanding-window temporal cross-validation with a uniform, performance-independent fold-inclusion criterion that prevents policies trained on attack-scarce periods from being scored;
- an imbalance-aware benchmark of *seven* offline policy-learning agents spanning imitation (memoryless and recurrent behavioral cloning), reward-weighted recurrent, value-based, and return-conditioned sequence-modeling families for SOC alert triage, evaluated on two identifier-complete, feature-rich datasets (UNSW-NB15 and CIC-IDS-2017) with detection, policy-recovery, and SOC-response metrics under Friedman, Nemenyi, and Wilcoxon tests;
- a controlled recurrence-versus-reward analysis: by adding a reward-free recurrent imitator (Recurrent Behavioral Cloning, $w_t \equiv 1$) and comparing it against the memoryless imitator and the advantage-weighted recurrent policy, we isolate the source of the gains — recurrence and trajectory context, not reward-derived advantage weighting — and show that the reward-free recurrent imitator is the strongest agent on both datasets;
- a multi-seed, multi-fold stability analysis showing that value-based methods and the Decision Transformer degenerate into trivial detectors — a collapse concealed by F1 and ROC-AUC but exposed by the Matthews correlation coefficient — which establishes multi-metric, stability-aware reporting as a first-class evaluation criterion for offline learning in security; and
- a demonstration that the resulting recurrent policies are dependable yet inherently conservative, deployed in an advisory, human-in-the-loop mode with a deterministic severity floor so that no medium- or high-severity alert can be auto-dismissed and containment remains analyst-owned.

The remainder of the paper is organized as follows. Section 2 formalizes the problem; Section 3 details the methodology; Section 4 describes the experimental setup; Section 5 reports results; Section 5.9 discusses the findings; Section 6 highlights safety-gated deployment considerations; Section 7 outlines the threats to validity and our mitigation strategies; and Section 8 concludes.

2. Problem formulation

Table 1 lists the notation used throughout. We model alert triage as a finite-horizon Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ learned purely from a fixed, logged dataset $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$ with no environment interaction. The action space $\mathcal{A} = \{0, 1, 2\}$ encodes the escalating response levels (Dismiss, Investigate, Contain), and the reward r_t trades threat detection against false-positive and operational-disruption costs.

Definition 1 (*Leakage-free Temporal Evaluation*)**.** Let $\mathrm{time}(\cdot)$ denote flow timestamp. A split into $(\mathcal{D}_{\mathrm{tr}}, \mathcal{D}_{\mathrm{va}}, \mathcal{D}_{\mathrm{te}})$ is *leakage-free* if $\max_{x\in\mathcal{D}_{\mathrm{tr}}} \mathrm{time}(x) \le \min_{x\in\mathcal{D}_{\mathrm{va}}} \mathrm{time}(x)$ and $\max_{x\in\mathcal{D}_{\mathrm{va}}} \mathrm{time}(x) \le \min_{x\in\mathcal{D}_{\mathrm{te}}} \mathrm{time}(x)$, and every preprocessing statistic is estimated on $\mathcal{D}_{\mathrm{tr}}$ only.

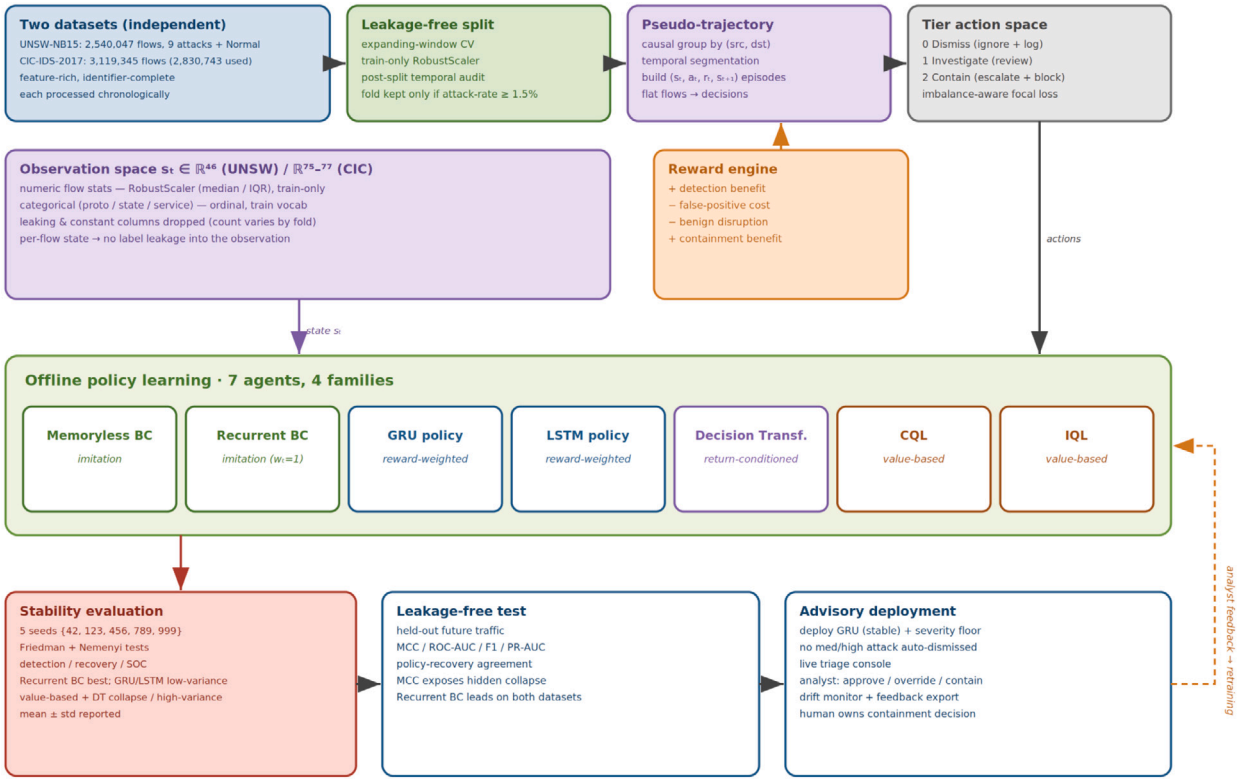


Fig. 1. The SOC-RL pipeline. Raw flows from two feature-rich, identifier-complete intrusion datasets — UNSW-NB15 and CIC-IDS-2017, with per-flow observations $s_t \in \mathbb{R}^{46}$ and $s_t \in \mathbb{R}^{75-77}$ respectively — each undergo a leakage-free chronological split under expanding-window temporal cross-validation, with a uniform, performance-independent fold-inclusion criterion (training attack rate $\geq 1.5\%$). Pseudo-trajectory reconstruction then converts each flat flow table into decision episodes (s_t, a_t, r_t, s_{t+1}): the observation s_t is the scaled flow-feature vector, the action a_t is drawn from the analyst-tier space (DISMISS, INVESTIGATE, CONTAIN), and the reward r_t balances threat detection against false-positive and operational-disruption costs. Seven offline policy-learning agents — memoryless and recurrent imitation baselines (Behavioral Cloning), reward-weighted recurrent sequence policies (GRU, LSTM), value-based offline RL (CQL, IQL), and a return-conditioned sequence model (Decision Transformer) — are trained on the reconstructed episodes, evaluated under a stability-aware multi-seed protocol with Friedman and Nemenyi tests on a leakage-free future-test fold, and deployed in advisory, human-in-the-loop mode in which containment remains analyst-owned.

The seven offline policy-learning agents we benchmark share this dataset and MDP structure but optimize distinct objectives that differ in *how* the reward enters training. The value-based offline RL agents CQL and IQL seek a policy π_θ that maximizes the expected discounted return $\mathbb{E}\left[\sum_t \gamma^t r_t\right]$ while suppressing value over-estimation on out-of-distribution actions, which keeps the learned policy close to the data-generating behavior policy π_β. The Decision Transformer casts control as return-conditioned sequence modeling, consuming the reward as a target return-to-go at both training and inference. Behavioral cloning is the one reward-free agent, maximizing the likelihood of the logged analyst action a_t given s_t by supervised learning; the recurrent GRU and LSTM policies retain this supervised action-prediction objective but reweight it by an advantage derived from returns-to-go (Section 3.6), so they use the reward *indirectly*, as a weight on imitation rather than through value bootstrapping. The reward function applies to all seven agents at evaluation, where the discounted return of every learned policy is scored under r_t (Section 4.5), placing all methods on a common axis.

Table 2 situates the four offline policy-learning families evaluated in this study, summarizing how each uses the reward and its principal strengths and limitations for SOC triage.

3. Methodology

3.1. Datasets

1. **UNSW-NB15:** This dataset contains approximately 2.54 million network flows, an attack prevalence of roughly 12.6%, and 10 traffic classes (normal traffic plus nine attack categories). UNSW-NB15 is a modern network intrusion-detection benchmark developed by the Australian Centre for Cyber Security (ACCS) using the IXIA PerfectStorm framework to generate realistic benign traffic and contemporary cyberattack scenarios within a controlled cyber-range environment (Moustafa et al., 2017; Moustafa & Slay, 2015). Raw network traffic was captured in `pcap` format (approximately 100 GB) and subsequently processed using tools such as Argus and Bro-IDS to derive 49 engineered flow-based features with corresponding attack labels. The dataset contains normal traffic alongside nine representative attack categories: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. Overall, UNSW-NB15 comprises 2,540,047 labeled network-flow records; although the benchmark ships with predefined train/test partitions, we use the complete record set and impose our own leakage-free chronological splits (Section 3.3), rather than the shipped partition. It is a widely adopted benchmark for evaluating machine-learning and deep-reinforcement-learning approaches to cybersecurity intrusion detection and SOC-oriented threat analysis.
2. **CIC-IDS-2017:** The second evaluation dataset is CIC-IDS-2017,[1] a flow-based intrusion-detection benchmark generated by the Canadian Institute for Cybersecurity over five consecutive working days (Monday–Friday). We use the identifier-complete release, which — unlike the widely redistributed machine-learning variants that strip host addresses — retains the source and destination IP addresses

[1] https://www.kaggle.com/datasets/pshikk/cicids2017-untampered.

Table 1
Notation used throughout the analysis.

Symbol	Meaning
$\mathcal{M}$	Finite-horizon Markov decision process
$\mathcal{S}$	State/observation space
$\mathcal{A}$	Action space, $\{0, 1, 2\}$ (Dismiss/Investigate/Contain)
P	Transition dynamics of the MDP
r	Reward function; r_t is the reward at step t
γ	Discount factor
s_t	Observation (scaled flow-feature vector) at step t
a_t	Analyst triage action at step t
r_t	Multi-objective reward at step t
s_{t+1}	Next observation following action a_t
$\mathcal{D}$	Offline dataset of logged/reconstructed transitions
$\mathcal{D}_{\text{tr}}$	Training split of the offline dataset
$\mathcal{D}_{\text{va}}$	Validation split of the offline dataset
$\mathcal{D}_{\text{te}}$	Test split of the offline dataset
x	Transition/sample used for temporal ordering
time(x)	Flow timestamp associated with sample x
w_t	Per-token imitation weight ($w_t \equiv 1$ for imitation; advantage otherwise)
G_t	Return-to-go from step t; $\overline{G}$ is its batch mean
c_a	Inverse-frequency class weight for action a
γ_f	Focal-loss focusing parameter
τ	Inter-arrival temporal gap threshold for episode segmentation
$L_{\min}, L_{\max}$	Minimum and maximum episode length
m_t	Sequence-validity mask, $m_t \in \{0, 1\}$
π_θ	Learned policy; π_β is the behavior policy
MCC	Matthews correlation coefficient (primary detection metric)

Following the conventions of the original methods, τ additionally denotes the expectile parameter of IQL and $\tau_{<t}$ the preceding sub-trajectory consumed by the Decision Transformer; both are local to those agents' objectives and are disambiguated by context. Flow timestamps are written time$(\cdot)$ throughout, matching Algorithms 3–5.

Table 2
Offline policy-learning families: how each uses the reward, with strengths and limitations for SOC alert triage.

Family (agents)	Reward use	Strengths	Limitations
Imitation/BC (Memoryless BC, Recurrent BC)	None (reward-free)	Stable; faithfully reproduces analyst behavior; strongest here	Cannot exceed demonstrated returns by construction
Adv.-weighted recurrent (GRU, LSTM)	Advantage weight on a supervised loss	Trajectory-aware; retains an imitation anchor	Advantage weighting adds seed variance; little gain over reward-free recurrence
Value-based offline RL (CQL, IQL)	TD value bootstrapping	Principled conservatism for OOD actions	Collapse to trivial detectors under severe imbalance
Return-conditioned (Decision Transformer)	Return-to-go conditioning	Flexible sequence model	High variance; non-generalizing; most expensive

and the per-flow timestamp required by our causal grouping and leakage-free temporal split. Each flow is described by approximately eighty CICFlowMeter statistics spanning packet-length, inter-arrival-time, flag-count, and flow-rate features, placing CIC-IDS-2017 in the same feature-rich regime as UNSW-NB15. After concatenating the eight capture files (several days are split across multiple CSVs), the raw corpus comprises 3,119,345 flows; after removing flows with invalid feature values and those that cannot be causally grouped, 2,830,743 flows are retained for reconstruction and evaluation. Attacks are distributed across the week by type: brute-force (FTP-/SSH-Patator) on Tuesday; denial-of-service variants and Heartbleed on Wednesday; web attacks and infiltration on Thursday; and botnet, port-scan, and distributed denial-of-service activity on Friday. This day-segregated structure is a defining property of the dataset and has a direct consequence for temporal evaluation, discussed below.

3.2. Temporal splitting protocol

Both datasets are split under the same scheme. All records are first sorted chronologically by flow start time. Rather than rely on a single chronological partition, we adopt five-fold expanding-window temporal cross-validation: the timeline is divided into successive blocks and, at each fold, the model trains on all flows up to a rolling origin and is evaluated on the subsequent block, with an intervening validation block. This yields the leakage-free split of Definition 1 at every fold — each training flow precedes its validation flows, which in turn precede its test flows — while assessing sensitivity to the particular period on which a model is evaluated. Crucially, all preprocessing statistics — feature scaling in particular — are re-estimated on each fold's training partition only and applied unchanged to its validation and test partitions, so that no information from the future leaks into the past.

UNSW-NB15. As the origin advances across folds, the training window grows (from 355,606 to 1,778,032 flows) and the test window shrinks, so that later folds are trained on more history, mirroring how a deployed detector accumulates data over time. Table 3 reports the resulting per-fold composition. Two properties merit emphasis. First, because the partitions are purely chronological rather than stratified, the attack rate is not constant across folds: training attack rates vary with the period covered (0.021–0.097), while every test block retains a substantial attack rate (0.142–0.214), confirming that no fold is attack-starved. This temporal drift is a faithful reflection of how the dataset was collected and of how a deployed detector encounters non-stationary traffic; a stratified split would erase it at the cost of reintroducing temporal leakage, which we deliberately avoid. Second, several attack families (e.g. Worms, Shellcode) are extremely rare in every fold, a sparsity that directly motivates the imbalance-aware training objective introduced in Section 3.6. Folds 1 and 2 have all-benign validation blocks (attack rate 0.000); for these folds, model

Table 3
Chronological expanding-window temporal cross-validation splits on UNSW-NB15 (train/validation/test flow counts and attack rates). Splits are strictly chronological: training always precedes validation, which precedes test (leakage-free), and as the rolling origin advances the train window grows while the test window shrinks. The fold-inclusion criterion (Section 3.3) is evaluated on the *training* attack rate only, since it governs whether a fold contains enough positive examples to fit a policy; all five folds qualify here (training attack rate 0.021–0.097). Validation is used solely for model selection. Every test block retains a substantial attack rate (0.14–0.21), confirming that no fold is attack-starved. Counts are raw partition flows; the reconstructed transition counts actually scored and the retained percentage are reported in Table 5.

Fold	Train		Validation		Test	
	Flows	Attack rate	Flows	Attack rate	Flows	Attack rate
1	355,606	0.062	76,201	0.000	2,108,240	0.142
2	711,212	0.031	152,403	0.000	1,676,432	0.178
3	1,066,819	0.021	228,604	0.144	1,244,624	0.214
4	1,422,425	0.057	304,806	0.257	812,816	0.200
5	1,778,032	0.097	381,007	0.186	381,008	0.204

Table 4
Chronological expanding-window temporal cross-validation splits on CIC-IDS-2017. Training always precedes validation, which precedes test in time (leakage-free); as the rolling origin advances the training window grows and the test window shrinks. Per our inclusion criterion (Section 3.3), a fold is evaluated only when its *training* attack rate reaches at least 1.5%. Folds 1–3 fall below this threshold (training attack rate ≤ 0.012) owing to the day-segregated attack schedule and are excluded; folds 4–5 (training attack rate 0.135–0.162) are retained. Every test block retains a substantial attack rate (0.237–0.321) regardless of inclusion. Counts are raw partition flows; reconstructed transition counts and the retained percentage are reported in Table 5.

Fold	Train		Validation		Test		Eval?
	Flows	Attack rate	Flows	Attack rate	Flows	Attack rate	
1	396,303	0.000	84,922	0.000	2,349,518	0.237	✗
2	792,607	0.009	169,845	0.039	1,868,291	0.291	✗
3	1,188,911	0.012	254,767	0.455	1,387,065	0.309	✗
4	1,585,215	0.162	339,689	0.031	905,839	0.321	✓
5	1,981,520	0.135	424,611	0.378	424,612	0.307	✓

selection uses a criterion that does not require validation positives, so early stopping remains well defined.

CIC-IDS-2017: temporal split and attack-type shift. CIC-IDS-2017 is split under the identical five-fold expanding-window protocol (Table 4). As the origin advances, the training window grows from 396,303 to 1,981,520 flows and the test window shrinks from 2,349,518 to 424,612 flows. Because attacks in CIC-IDS-2017 are segregated by day and the capture opens with an all-benign period, the splits exhibit a pronounced temporal shift in both attack *rate* and attack *type*. The training attack rate rises from 0.000 in the earliest fold — whose training window contains only benign Monday traffic — through 0.009 and 0.012 as brute-force and a handful of other flows enter, to 0.162 and 0.135 in the two mature folds (folds 4 and 5) once denial-of-service days are absorbed; the slight dip in the final fold reflects the larger benign base accumulated by the widest expanding window. Every test block, by contrast, retains a substantial attack rate (0.237–0.321), so no fold is attack-starved on the evaluation side. The attack types likewise differ across the timeline: denial-of-service and brute-force traffic appear first, port-scanning and web attacks later, and distributed denial-of-service and botnet activity last, so that earlier folds are tested largely on attack types unseen during training. Under a strict leakage-free protocol this shift is not a deficiency to be corrected by shuffling — doing so would reintroduce the temporal leakage the protocol is designed to prevent — but a genuine property of the data. CIC-IDS-2017 therefore probes two stresses at once: sensitivity to the evaluation period (via the multi-fold design) and generalization across attack categories (via the day-segregated attack types), complementing the more evenly distributed UNSW-NB15.

Flow counts versus reconstructed transitions. Tables 3 and 4 report the *raw* chronological partition sizes in flows. The offline agents are not trained on raw flows but on the reconstructed pseudo-trajectory *transitions*, which are fewer for two reasons intrinsic to the reconstruction (Section 3.2): causal grouping on $(\text{srcip}, \text{dstip})$ drops singleton pairs below the minimum group size, and temporal segmentation discards episodes shorter than the minimum length. Consequently the number of decision points actually scored is below the flow count of the corresponding partition. On CIC-IDS-2017 the two included folds yield $1{,}292{,}110$ and $1{,}609{,}248$ training transitions and 751,406 and 356,399 test transitions (from $1{,}585{,}215/1{,}981{,}520$ training and $905{,}839/424{,}612$ test flows, i.e. 81%–84% retained). On UNSW-NB15 the final fold — used for the single-fold ablation, reward-sensitivity, and deployment analyses — reconstructs to $1{,}700{,}685$ training transitions (48,813 episodes) from $1{,}778{,}032$ flows, 355,492 validation transitions (13,183 episodes) from 381,007 flows, and 360,227 test transitions (11,630 episodes) from the 381,008-flow test block (95.6%, 93.3%, and 94.6% retained). The $\approx 20{,}781$ test flows not scored fall in the 9124 episodes shorter than the minimum episode length that the segmenter discards on this split. UNSW-NB15 retains more than CIC-IDS-2017 because its flows form longer host sessions, whereas many CIC-IDS-2017 scan and denial-of-service flows are isolated and fall below the grouping or length thresholds. The all-metric results in Section 5 are computed over these reconstructed transitions.

3.3. Fold-inclusion criterion

Because the expanding-window protocol advances a rolling origin through chronologically ordered flows, the earliest folds may cover periods containing few labeled attacks. A fold whose training window holds too few positive examples cannot support learning a triage policy; evaluating on it would measure an untrained model rather than genuine temporal generalization. We therefore adopt a uniform inclusion criterion, fixed in advance and independent of any model's performance: *a fold is evaluated only if its training-window attack rate is at least 1.5%.* This threshold is defined solely by the split's attack coverage, not by any observed result, and is applied identically to every dataset. Under it, all five UNSW-NB15 folds qualify (training attack rate 0.021–0.097;

Table 5
Pseudo-trajectory reconstruction retention on the evaluated folds. Raw chronological flow partitions are reconstructed into decision transitions, on which agents are trained and evaluated. Across the training and test partitions of the evaluated folds, UNSW-NB15 retains 94.0%–96.8% of flows, whereas CIC-IDS-2017 retains 81.2%–83.9%. The lower retention for CIC-IDS-2017 is consistent with a larger proportion of isolated scan and denial-of-service flows that do not satisfy the minimum episode-length requirement. Values were verified from the run logs.

Dataset	Fold	Train			Test		
		Flows	Trans.	Kept	Flows	Trans.	Kept
UNSW-NB15	1	355,606	338,575	95.2%	2,108,240	2,003,508	95.0%
	2	711,212	685,638	96.4%	1,676,432	1,581,965	94.4%
	3	1,066,819	1,033,032	96.8%	1,244,624	1,169,399	94.0%
	4	1,422,425	1,364,583	95.9%	812,816	764,135	94.0%
	5	1,778,032	1,700,685	95.6%	381,008	360,227	94.5%
CIC-IDS-2017	4	1,585,215	1,292,110	81.5%	905,839	751,406	83.0%
	5	1,981,520	1,609,248	81.2%	424,612	356,399	83.9%

Table 3), whereas on CIC-IDS-2017 — where attacks are segregated by capture day and the recording opens with an all-benign period — folds 1–3 fall below the threshold (training attack rate ≤ 0.012) and only the mature-window folds 4–5 are evaluated (Table 4). Consequently, the CIC-IDS-2017 stability estimates rest on two qualifying folds against UNSW-NB15's five; this asymmetry is an intended consequence of the leakage-free design rather than a tuning choice, and we account for it when interpreting cross-dataset stability in Section 5.

3.4. Pseudo-trajectory reconstruction

Sequence-based offline policy learning requires sequential decision trajectories, yet public intrusion datasets provide only independent flow records with no notion of an analyst's evolving response. We bridge this gap through a *pseudo-trajectory reconstruction* pipeline composed of causal grouping, temporal segmentation, and transition generation (Algorithms 1–3). The overall workflow is summarized in Algorithm 1, while the grouping and segmentation stages are specified in Algorithms 2 and 3, respectively. The recurrent and Decision-Transformer agents consume this sequential structure directly; behavioral cloning uses the same episodes as unordered (s, a) pairs.

First, flows are *causally grouped* using CausalGroup (Algorithm 2), which partitions the chronologically ordered flow table according to the (source IP, destination IP) pair. The underlying premise is that flows exchanged between the same host pair constitute a coherent interaction whose handling should be considered jointly. The grouping procedure performs a single chronological pass over the dataset, preserves temporal ordering within each group, removes undersized groups, and returns a deterministic set of candidate interaction traces. Second, each group is *temporally segmented* into bounded episodes through TemporalSegment (Algorithm 3). Segmentation boundaries are introduced using three complementary rules:

1. inter-arrival gap thresholds, which separate temporally distant activities;
2. attack-stage transitions, identified through changes in category labels and used only for structural partitioning; and
3. maximum-length constraints, which prevent prolonged interactions from collapsing into unbounded trajectories.

Finally, minimum-length filtering removes degenerate episodes and ensures that the resulting trajectories contain sufficient sequential context for the sequence-based agents.

Third, within each segmented episode, Algorithm 1 constructs transitions (s_t, a_t, r_t, s_{t+1}). The state s_t corresponds to the scaled feature vector of the current flow, the action a_t represents the triage decision recorded by the behavior policy, the reward r_t is assigned by the reward engine $\mathcal{R}$ described in Section 3.5, and s_{t+1} denotes the subsequent flow in the same segment; the final flow of each segment yields a terminal transition with no successor state. The resulting episode collection $\mathcal{E}$ forms the offline dataset D introduced in Section 2. Because the behavior policy is reconstructed directly from dataset labels, the resulting trajectories support evaluation of *policy recovery* rather than novel-attack discovery. The methodological scope and its implications are described in Section 7.

Algorithm 1 Pseudo-trajectory reconstruction

Require: Chronologically ordered flows F, reward engine $\mathcal{R}$
$G \leftarrow$ CausalGroup(F) ▷ group by (srcip, dstip)
$\mathcal{E} \leftarrow \emptyset$
for each group $g \in G$ **do**
  segments $\leftarrow$ TemporalSegment(g) ▷ gap / stage / length rules
  for each segment $e \in$ *segments* **do**
    build transitions (s_t, a_t, r_t, s_{t+1}) with $r_t = \mathcal{R}(\cdot)$; mark last flow of e terminal
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{e\}$
  end for
end for
return episode set $\mathcal{E}$

Algorithm 2 CausalGroup: causal flow grouping

Require: Chronologically ordered flows F; grouping key κ (default (srcip, dstip)); minimum group size $m_{\min}$
Ensure: Time-ordered candidate groups G
assert F is sorted ascending by timestamp ▷ precondition for valid reconstruction
$H \leftarrow \emptyset$ ▷ map: key value → ordered flow list
for each flow $f \in F$ **do** ▷ single pass preserves time order
  $k \leftarrow \kappa(f)$ ▷ e.g. (srcip, dstip) pair
  append f to $H[k]$
end for
$G \leftarrow \{ g \in H : |g| \geq m_{\min} \}$ ▷ discard singletons / tiny groups
sort G by each group's earliest flow time ▷ deterministic downstream order
return G

3.5. Reward engineering

The reward function encodes the operational trade-offs that a security operations center (SOC) balances during triage: catching genuine threats while avoiding both alert fatigue and the business disruption of over-aggressive containment. For each transition, we define a multi-objective reward (Eq. (1)) as a weighted combination of four terms: a *detection* benefit ρ_{detect} that rewards escalating true threats, a *false-positive* penalty ρ_{fp} that discourages flagging benign traffic, a *benign-disruption* penalty ρ_{disrupt} that specifically charges for containing benign flows, and a *containment* benefit ρ_{contain} that credits correctly

Algorithm 3 TemporalSegment: gap/stage/length episode segmentation

Require: Time-ordered group $g = \langle f_1, \dots, f_n \rangle$; gap bound τ; max length $L_{\max}$; min length $L_{\min}$; stage flag *useStage*
Ensure: Bounded episodes *segments*

```
segments ← ∅;  e ← ⟨f_1⟩                                   ▷ current episode
for i ← 2 to n do
    Δ ← time(f_i) − time(f_{i−1})                          ▷ inter-arrival gap
    if Δ > τ then                                          ▷ temporal-gap boundary
        reason ← GAP
    else if useStage and cat(f_i) ≠ cat(f_{i−1}) then
        reason ← STAGE          ▷ attack-stage change (labels used for structure only)
    else if |e| ≥ L_max then                               ▷ length cap
        reason ← LENGTH
    else
        reason ← ∅
    end if
    if reason ≠ ∅ then
        if |e| ≥ L_min then segments ← segments ∪ {e}
        e ← ⟨f_i⟩                                          ▷ start new episode
    else
        append f_i to e
    end if
end for
if |e| ≥ L_min then segments ← segments ∪ {e}              ▷ flush final episode
return segments
```

neutralizing confirmed attacks.

$$r_t = \underbrace{w_d\,\rho_{\text{detect}}}_{\text{detection}} - \underbrace{w_f\,\rho_{\text{fp}}}_{\text{false positive}} - \underbrace{w_b\,\rho_{\text{disrupt}}}_{\text{benign disruption}} + \underbrace{w_c\,\rho_{\text{contain}}}_{\text{containment}}. \tag{1}$$

The non-negative weights w_d, w_f, w_b, w_c govern the relative importance of detection against the cost of disruption; raising the penalty terms yields a more conservative policy, while raising the benefit terms yields a more aggressive one. Rewards are normalized across the training set before agent optimization to stabilize value estimation. During training the reward is used by every agent except behavioral cloning: through the Bellman target for CQL and IQL, through return-to-go conditioning for the Decision Transformer, and through advantage weighting of the supervised loss for the recurrent GRU and LSTM policies (Section 3.6). Behavioral cloning alone is reward-free during fitting. The reward nonetheless applies to all agents at evaluation, where the discounted return of every learned policy is scored under r_t (Section 4.5), placing all methods on a common axis.

3.6. Offline policy-learning agents

We benchmark seven offline policy-learning agents grouped into four families, distinguished by *how* the reward enters training and ordered here along the imitation–RL spectrum. (i) The *imitation baseline*, behavioral cloning (BC), learns only from state–action pairs and is the one agent that never uses the reward. (ii) The *reward-weighted sequence policies*, our recurrent GRU and LSTM agents, retain a supervised sequence-modeling objective but reweight it by a reward-derived advantage, so they use the reward as a weight on imitation rather than as a bootstrap or generation target. (iii) The *value-based offline RL* agents, CQL and IQL, use the reward through temporal-difference value learning. (iv) The *return-conditioned sequence model*, the Decision Transformer, consumes the reward as a target return-to-go that conditions autoregressive action generation. Thus five of the seven agents use the reward during training, through different mechanisms; the two imitation agents — memoryless and recurrent Behavioral Cloning — are reward-free. A useful axis for interpreting the results (Section 5) is whether an agent retains a supervised action-prediction anchor: BC (pure cross-entropy) and the reward-weighted sequence policies (advantage-reweighted cross-entropy) do, whereas the value-based agents and the Decision Transformer replace it with value bootstrapping or return-conditioned generation.

3.6.1. Imitation baseline

Behavioral Cloning (BC). BC is an imitation-learning method that learns a policy by directly reproducing the actions recorded in a fixed dataset of historical decisions (Pomerleau, 1988; Ross et al., 2011). Unlike reinforcement learning, BC does not interact with an environment or optimize cumulative reward; instead it formulates policy learning as a supervised classification problem in which the agent predicts actions from observed states. Given a dataset of state–action pairs (s_t, a_t), the policy $\pi_\theta(a_t \mid s_t)$ is trained to maximize the likelihood of demonstrated actions. In this implementation, the `BehaviorCloningAgent` employs a neural-network policy trained with cross-entropy loss, and inverse-frequency class weighting is applied to mitigate action imbalance. BC serves both as a baseline for the reward-using agents benchmarked here and as a warm-start initialization for the conservative value-based agents. Because BC only reproduces observed behavior and does not use reward information, it cannot by construction exceed the returns present in the demonstrations—a limitation that makes its competitiveness in Section 5 all the more notable.

$$L^{\text{BC}}(\theta) = -\mathbb{E}_{(s_t,a_t)\sim D}\left[\log \pi_\theta(a_t \mid s_t)\right] \tag{2}$$

where D denotes the offline dataset of reconstructed demonstrations, s_t the observed state at time t, and a_t the corresponding demonstrated action. The policy outputs a distribution over actions, and training minimizes the negative log-likelihood of the demonstrated behavior.

3.7. Recurrent behavioral cloning

Memoryless Behavioral Cloning maps each flow independently, discarding the sequential context that the pseudo-trajectory reconstruction makes available. To separate the contribution of *recurrence* from that of *reward engineering*, we introduce a recurrent imitation agent, Recurrent Behavioral Cloning, that conditions on the trajectory prefix through a gated recurrent unit (GRU) but, unlike the advantage-weighted recurrent policy, uses no reward-derived weighting. Given a reconstructed episode $(s_1, a_1, \dots, s_T, a_T)$ with valid-step mask $m_t \in \{0, 1\}$, it minimizes a masked, class-weighted focal cross-entropy on the behavior-policy actions,

$$\mathcal{L} = \frac{\sum_{t=1}^{T} m_t\, w_t\, c_{a_t}\left(1 - \pi_\theta(a_t \mid s_{\le t})\right)^{\gamma_f}\left[-\log \pi_\theta(a_t \mid s_{\le t})\right]}{\sum_{t=1}^{T} m_t}, \tag{3}$$

where $\pi_\theta(a_t \mid s_{\le t})$ is the policy's probability of the analyst action a_t given the recurrent encoding of the prefix $s_{\le t}$, c_{a_t} the inverse-frequency class weight, and γ_f the focal focusing parameter. Recurrent Behavioral Cloning fixes the per-token weight $w_t \equiv 1$; the advantage-weighted recurrent policy of Section 3.6 instead sets $w_t = \max(G_t - \overline{G}, 0) + \epsilon$ with return-to-go G_t, so the two share an identical architecture and differ *only* in whether the reward enters the objective. Recurrent Behavioral Cloning thus differs from Memoryless Behavioral Cloning solely in conditioning ($s_{\le t}$ vs. s_t), isolating trajectory context, and from the advantage-weighted policy solely in the reward term, isolating reward engineering.

3.7.1. Reward-weighted sequence policies

The GRU and LSTM agents form a family of their own: recurrent sequence policies whose supervised action-prediction loss is reweighted by a reward-derived advantage (advantage-weighted regression). They are neither pure imitation — they consume the reward — nor value-based offline RL, since they never bootstrap a value function or condition generation on a target return. Instead the reward enters only as a per-transition weight on an otherwise supervised sequence loss, which keeps them anchored to the demonstrated behavior while biasing learning toward higher-return trajectories.

Gated Recurrent Unit (GRU) policy. The GRU policy captures temporal dependencies in the reconstructed triage episodes. Unlike memoryless behavioral cloning, which maps actions from the current state alone, the GRU policy conditions action selection on the trajectory history through a recurrent hidden representation:

$$a_t \sim \pi_\phi(a_t \mid s_t, s_{t-1}, \dots, s_1), \tag{4}$$

where the hidden state recursively summarizes prior observations up to step t. The GRU network processes offline trajectories and outputs action logits through a linear prediction layer, optimized by a sequence-level cross-entropy (behavioral-cloning) loss *reweighted by a reward-derived advantage*:

$$\mathcal{L}_{\text{GRU}} = -\mathbb{E}_{(s_t,a_t)\sim D}\big[\, w_t \, \log \pi_\phi(a_t \mid h_t)\,\big], \tag{5}$$

$$w_t = \max\big(0,\ \hat{R}_t - \bar{R}\,\big) + 0.1,$$

where h_t is the GRU hidden representation at step t, $\hat{R}_t$ the normalized return-to-go from step t (a reward-derived quantity), and $\bar{R}$ its batch mean. The advantage weight w_t up-weights transitions drawn from higher-return trajectories while leaving the supervised action-prediction objective intact; setting $w_t \equiv 1$ recovers pure recurrent behavioral cloning, so the family degrades gracefully to imitation as the advantage signal vanishes. The reward is used, but only as a weight on a supervised sequence loss (advantage-weighted regression), never through value bootstrapping or return conditioning. The update and reset gates capture short- and medium-term dependencies at lower computational cost than an LSTM, making the GRU policy well suited to SOC environments where evolving alert streams and attack-progression patterns require temporally aware decisions.

Long Short-Term Memory (LSTM) policy. The LSTM policy (Hochreiter & Schmidhuber, 1997) is the second reward-weighted sequence policy, designed to learn longer-range temporal dependencies than the GRU. As with the GRU policy, it conditions actions on the sequence history rather than the current state alone, and it uses the identical advantage-weighted objective. The LSTM processes trajectory sequences through memory cells regulated by input, output, and forget gates, enabling stable long-term retention across extended episodes, and is trained by the same reward-reweighted sequence-level cross-entropy loss as the GRU policy (Eq. (5)), with h_t the latent state produced by the LSTM encoder. Its explicit memory cell makes it particularly suited to prolonged attack chains, multi-stage intrusions, and long-horizon dependencies requiring persistent contextual reasoning.

3.7.2. Value-based offline RL

Conservative Q-Learning (CQL). CQL (Kumar et al., 2020) is an offline RL algorithm designed to address over-estimation of out-of-distribution actions in static datasets. In offline settings the agent cannot interact with the environment to verify whether unseen actions yield high rewards, leading standard Q-learning to exploit inaccurate Q-values for rarely observed actions. CQL mitigates this by learning a conservative action-value function, penalizing high Q-values for unsupported actions while preserving high values for actions observed in the data. In this implementation, the `CQLAgent` optimizes a standard Bellman error together with a conservative regularization term based on the discrete log-sum-exp formulation. A target network stabilizes temporal-difference learning, and optional BC warm-start initialization biases the Q-network toward the behavior policy before conservative fine-tuning. The conservatism coefficient α controls how strongly unseen or low-support actions are penalized.

$$\begin{aligned} L^{\text{CQL}}(\theta) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim D}\Big[\big(Q_\theta(s_t,a_t) - (r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1},a'))\big)^2\Big] \\ + \ \alpha\, \mathbb{E}_{s_t\sim D}\Big[\log \textstyle\sum_a \exp(Q_\theta(s_t,a)) - Q_\theta(s_t,a_t)\Big] \end{aligned} \tag{6}$$

Here s_t is the state at step t, a_t the action, r_t the immediate reward, s_{t+1} the next state, $\gamma \in (0,1)$ the discount factor, Q_θ the learned action-value function, Q_{θ^-} the delayed target network, and $\alpha > 0$ the conservatism coefficient; a_t inside the second expectation is the behavior (dataset) action.

Implicit Q-Learning (IQL). IQL (Kostrikov et al., 2021) is an offline RL algorithm that avoids out-of-distribution over-estimation without explicitly penalizing unseen actions. Unlike CQL, IQL never evaluates $\max_{a'} Q(s',a')$, which can destabilize learning on static datasets. Instead it decomposes value learning into three coupled components: (i) an expectile-based state-value function, (ii) a Q-function regressed toward bootstrapped state values, and (iii) an advantage-weighted policy-extraction step. The `IQLAgent` learns $V_\psi(s_t)$ by expectile regression over dataset actions, trains the Q-function against a one-step Bellman backup using $V_\psi(s_{t+1})$ in place of an action maximization, and extracts the policy by advantage-weighted regression, so that learning stays fully within the support of D.

$$\begin{aligned} L^{\text{IQL}}(\theta,\psi,\phi) = & \underbrace{\mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim D}\big[(Q_\theta(s_t,a_t) - (r_t + \gamma(1-d_t)\,V_\psi(s_{t+1})))^2\big]}_{\text{Q-function Bellman regression}} \\ & + \underbrace{\mathbb{E}_{(s_t,a_t)\sim D}\big[\rho_\tau(Q_{\theta^-}(s_t,a_t) - V_\psi(s_t))\big]}_{\text{Expectile regression for } V_\psi} \\ & + \underbrace{-\mathbb{E}_{(s_t,a_t)\sim D}\big[\exp(\beta(Q_\theta(s_t,a_t) - V_\psi(s_t)))\log \pi_\phi(a_t|t_s)\big]}_{\text{Advantage-weighted policy learning}} \end{aligned} \tag{7}$$

where $s_t \in \mathcal{S}$, $a_t \in \mathcal{A}$, $r_t \in \mathbb{R}$, and $s_{t+1} \in \mathcal{S}$ are the transition elements, $d_t \in \{0,1\}$ marks termination, $\gamma \in (0,1)$ is the discount factor, V_ψ the expectile state-value function, π_ϕ the extracted policy, $\tau \in (0,1)$ the expectile parameter, $\beta > 0$ the advantage-weighting temperature, and $\rho_\tau(\cdot)$ the asymmetric expectile loss.

3.7.3. Return-conditioned sequence model

Decision Transformer (DT). DT (Chen et al., 2021) casts offline RL as return-conditioned sequence modeling rather than value-function optimization. Instead of estimating Q-values, DT autoregressively predicts actions conditioned on past trajectory context and a desired return-to-go (RTG). Unlike the recurrent policies, which reweight an imitation loss by an advantage, DT conditions its *generation* on a target return, consuming the reward as an input at both training and inference. For each step t,

$$\hat{R}_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}, \tag{8}$$

where γ is the discount factor and T the episode horizon. Over a context window of length K, the Transformer receives an interleaved sequence of RTG, state, and action tokens,

$$(\hat{R}_{t-K+1}, s_{t-K+1}, a_{t-K+1}, \dots, \hat{R}_t, s_t, a_t), \tag{9}$$

and predicts the next action autoregressively with a causal Transformer policy,

$$\pi_\theta(a_t \mid \tau_{<t}, \hat{R}_t) = \text{softmax}\big(f_\theta(\hat{R}_{t-K+1:t}, s_{t-K+1:t}, a_{t-K+1:t-1})\big), \tag{10}$$

where $\tau_{<t}$ is the preceding sub-trajectory and f_θ the Transformer action head. Training uses supervised sequence prediction with a class-weighted focal loss to counter SOC action imbalance,

$$\mathcal{L}_t = -\alpha_{a_t}(1-p_t)^{\gamma_f}\log p_t, \tag{11}$$

where p_t is the predicted probability of the true action, α_{a_t} the inverse-frequency class weight, and γ_f the focusing parameter. The objective averages masked token losses over valid positions,

$$\mathcal{L}(\theta) = \frac{1}{\sum_t m_t}\sum_t m_t \mathcal{L}_t, \tag{12}$$

with $m_t \in \{0,1\}$ the sequence-validity mask.

3.8. Leakage-free temporal evaluation protocol

The evaluation protocol is designed to eliminate the temporal leakage that inflates performance under conventional random splitting. It

operates *within each fold* of the expanding-window cross-validation of Section 3.2: the primitive in Algorithm 4 is instantiated once per rolling origin, taking that fold's chronological prefix as the flow pool F and producing its train, validation, and test partitions, so the safeguards below hold at every fold rather than for a single global split. Three safeguards enforce the leakage-free condition of Definition 1. First, the train, validation, and test partitions are strictly chronological, so that every evaluation flow occurs later in time than every training flow and no model is assessed on traffic it could have observed during fitting (Algorithm 4, lines 3–5). Second, all preprocessing statistics — most critically the feature scaler — are fitted on the training partition alone and applied unchanged to validation and test, preventing distributional information from the future from contaminating the past through the normalization step (Algorithm 4, lines 7–10). Third, a post-split leakage audit (Algorithm 5) verifies that the maximum training timestamp does not exceed the minimum validation timestamp, and likewise across the validation–test boundary, aborting the fold if either invariant is violated. On each fold's held-out test block we report the full metric battery of Section 4.5 and aggregate across the qualifying folds (Section 3.3), so that every agent is judged solely on genuinely future, unseen traffic.

Algorithm 4 Leakage-free temporal split (per expanding-window fold)

Require: Fold flow pool F with timestamps; split ratios $(\rho_{\text{tr}}, \rho_{\text{va}}, \rho_{\text{te}})$
Ensure: Scaled, leakage-free partitions $(D_{\text{tr}}, D_{\text{va}}, D_{\text{te}})$ for this fold

1: $F \leftarrow \textsc{SortByTime}(F)$ ▷ ascending flow start time
2: $N \leftarrow |F|$; $n_{\text{tr}} \leftarrow \lfloor \rho_{\text{tr}} N \rfloor$; $n_{\text{va}} \leftarrow \lfloor \rho_{\text{va}} N \rfloor$
3: $F_{\text{tr}} \leftarrow F[1 : n_{\text{tr}}]$ ▷ earliest traffic
4: $F_{\text{va}} \leftarrow F[n_{\text{tr}}{+}1 : n_{\text{tr}}{+}n_{\text{va}}]$
5: $F_{\text{te}} \leftarrow F[n_{\text{tr}}{+}n_{\text{va}}{+}1 : N]$ ▷ latest traffic
6: $\textsc{AuditTemporalOrder}(F_{\text{tr}}, F_{\text{va}}, F_{\text{te}})$ ▷ Alg. 5
7: $s \leftarrow \textsc{FitScaler}(F_{\text{tr}})$ ▷ statistics from train only
8: **for** $P \in \{F_{\text{tr}}, F_{\text{va}}, F_{\text{te}}\}$ **do**
9: $D_P \leftarrow \textsc{ApplyScaler}(s, P)$ ▷ same s for all splits
10: **end for**
11: **return** $(D_{\text{tr}}, D_{\text{va}}, D_{\text{te}})$

Algorithm 5 Post-split temporal leakage audit

Require: Ordered partitions $F_{\text{tr}}, F_{\text{va}}, F_{\text{te}}$
Ensure: Aborts if any temporal boundary is violated

1: $t_{\text{tr}}^{\max} \leftarrow \max_{f \in F_{\text{tr}}} \text{time}(f)$; $t_{\text{va}}^{\min} \leftarrow \min_{f \in F_{\text{va}}} \text{time}(f)$
2: $t_{\text{va}}^{\max} \leftarrow \max_{f \in F_{\text{va}}} \text{time}(f)$; $t_{\text{te}}^{\min} \leftarrow \min_{f \in F_{\text{te}}} \text{time}(f)$
3: **if** $t_{\text{tr}}^{\max} > t_{\text{va}}^{\min}$ **or** $t_{\text{va}}^{\max} > t_{\text{te}}^{\min}$ **then**
4: **abort** ▷ future traffic leaked into an earlier split
5: **end if**
6: **assert** $F_{\text{tr}} \cap F_{\text{va}} = \emptyset$ **and** $F_{\text{va}} \cap F_{\text{te}} = \emptyset$ ▷ scaler never refitted on val/test
7: **return** OK

3.9. Time and space complexity

The pseudo-trajectory reconstruction performs a single chronological pass over the N flows with hash-based causal grouping and linear segmentation, resulting in $O(N)$ time and $O(N)$ space complexity. For the recurrent policies (memoryless/recurrent BC, GRU, and LSTM), training and inference scale linearly with the number of transitions and the context length K, yielding $O(NK)$ time complexity and $O(K)$ recurrent-state memory per sequence; the memoryless imitator is the special case $K = 1$. The Decision Transformer's self-attention over its context window has $O(NK^2)$ time complexity and $O(K^2)$ attention-memory complexity. The value-based agents have computational costs that depend on the number of transitions processed per gradient step and the corresponding network architecture, as discussed above. These analytical complexity estimates are complemented by the measured wall-clock training, testing, and total times reported for each agent on both datasets (compute tables in Section 5), providing an empirical comparison of the computational costs of the evaluated approaches.

4. Experimental setup

4.1. Action space

We frame triage at the granularity of the standard SOC analyst-tier workflow, using a three-action space: Dismiss (close the alert without action), Investigate (route for analyst review), and Contain (apply a containment response). This tier model mirrors how a security operations center is staffed and escalated, and provides a benign dismissal floor appropriate to a setting in which the majority of alerts are closed without intervention. It is obtained by mapping the dataset's fine-grained dispositions onto the three tiers: passive handling (ignore, log) collapses to Dismiss; investigation maps to Investigate; and all escalation-and-above responses (escalate, block, quarantine, isolate) collapse to Contain. A flow is treated as flagged when the recommended action is Investigate or above.

4.2. Observation space

The observation $s_t \in \mathbb{R}^{46}$ is the feature vector of the network flow under consideration at decision step t. Each flow is represented by the connection-level attributes of UNSW-NB15 — including basic protocol fields, byte and packet counts, flow duration and timing statistics, and connection-state and content-based features — which together summarize the flow's behavior without reference to its ground-truth label. To prevent temporal leakage, all features are standardized with a robust scaler whose centering and scaling statistics are estimated on the training partition alone and applied unchanged to the validation and test partitions (Section 3.8); the median and interquartile range are used in place of the mean and standard deviation, so that the heavy-tailed distributions characteristic of network traffic do not distort the normalization. The agent therefore conditions its triage decision solely on observable flow characteristics, mirroring the information available to a SOC analyst at the moment an alert is raised.

4.3. Agents

We benchmark seven offline policy-learning agents spanning four methodological families. As an imitation baseline we use behavioral cloning (BC), which directly imitates the behavior policy and uses no reward. The reward-weighted sequence policies are recurrent GRU and LSTM agents that summarize trajectory history through a hidden state and reweight a supervised sequence loss by a reward-derived advantage (Section 3.6). From value-based offline RL, we include conservative Q-learning (CQL) (Kumar et al., 2020) and implicit Q-learning (IQL) (Kostrikov et al., 2021), which mitigate distributional shift through conservatism and expectile regression respectively. Finally, the Decision Transformer (DT) (Chen et al., 2021) is a return-conditioned sequence model that casts policy learning as return-conditioned action prediction. All agents share the reconstructed offline dataset and are trained without environment interaction.

Imbalance-aware objective. Because the behavior policy is dominated by passive actions while containment decisions are rare, a plain cross-entropy objective is overwhelmed by the common classes and fails to recover the rarer tiers. For the sequence-based agents (DT, GRU, LSTM) we therefore combine inverse-frequency class weighting with a focal loss (Lin et al., 2017), which down-weights well-classified common actions and concentrates gradient on the harder, under-represented ones. We use a focusing parameter of $\gamma_f = 1.5$.

Table 6
Ablation of recurrence and reward engineering (final fold, mean ± std over 5 seeds). Gains are relative to the row above.

Variant	MCC	F1	ΔMCC
Memoryless BC	0.493 ± 0.000	0.550 ± 0.000	–
Recurrent BC ($w_t \equiv 1$)	0.922 ± 0.003	0.935 ± 0.002	+0.429 (recurrence)
Advantage-weighted recurrent	0.864 ± 0.111	0.882 ± 0.100	−0.059 (reward eng.)

Table 7
Reward-weight sensitivity (final fold): MCC at the baseline reward and the range over the eight perturbations of four key coefficients.

Agent	Baseline MCC	Range over perturbations
Recurrent BC	0.911	[0.911, 0.911]
DT (return)	0.882	[0.411, 0.920]
GRU (adv.)	0.720	[0.613, 0.900]
LSTM (adv.)	0.607	[0.563, 0.747]
Memoryless BC	0.493	[0.493, 0.493]
CQL (value)	0.185	[−0.090, 0.309]
IQL (value)	−0.014	[−0.040, 0.015]

4.4. Implementation and hyperparameters

All agents are implemented in PyTorch and trained on the reconstructed training episodes. Unless otherwise noted, we use a hidden width of 64 with two layers, the Adam family of optimizers, a discount factor of $\gamma = 0.9$, and 6000 gradient steps. The Decision Transformer uses a context length of 25 transitions; the recurrent policies use a context length of 20 and an advantage-weighted objective (Eq. (5)). The deployed model is selected on the validation fold. Complete hyperparameter settings are provided in Appendix.

4.5. Evaluation metrics

We assess each agent along three complementary axes. *Detection quality* treats the policy as a binary threat detector — a flow is flagged when the action is INVESTIGATE or above — and is measured by precision, recall, F_1, ROC-AUC, PR-AUC, and Matthews correlation coefficient (MCC). Because the test attack rate is low, we treat MCC and ROC-AUC as the primary ranking metrics, as F_1 is sensitive to the operating threshold under class imbalance. *Policy recovery* measures how faithfully an agent reproduces the behavior-policy action, reported as both a frequency-weighted exact-match rate and an unweighted per-class mean (which weights rare tiers equally). *SOC response* captures operational timing and disposition, including mean time-to-detect (MTTD), mean time-to-respond (MTTR), and mean time-to-contain (MTTC) in episode steps, containment rate, decision correctness, and stability. Off-policy return is estimated by per-decision weighted importance sampling, with the dataset behavior return reported as a baseline.

4.6. Statistical protocol

To control for seed sensitivity, every agent is trained and evaluated across five random seeds $\{42, 123, 456, 789, 999\}$. We report means with standard deviations and assess cross-agent differences with the Friedman omnibus test, followed by Nemenyi post-hoc analysis, reporting a result as significant at $p < 0.05$.

4.7. Hardware specification

The experiments were carried out on a workstation running Ubuntu 24.04.3 LTS (64-bit), Linux kernel 6.17.0, powered by an Intel Core Ultra 9 275HX CPU (24 cores, 5.30 GHz), with 64 GB RAM and one NVIDIA GPU alongside Intel Integrated Graphics (Legion 9 18IAX10, 3840 × 2400).

5. Results and discussion

We evaluate on UNSW-NB15 under the leakage-free, expanding-window temporal protocol of Section 3, with the three-tier action space {DISMISS, INVESTIGATE, CONTAIN}. All agents train on the real PyTorch backend; no surrogate models are used. Unless a table is marked *final fold*, values are across-fold mean ± std over five expanding-window folds, each fold value being itself a five-seed mean, so the reported std is the *temporal sensitivity*: low std indicates robustness to the evaluation window. We present the component analyses first (what drives performance, the detection reference, how configurations are selected, and the policy-recovery reference) and establish the final benchmark verdict last.

5.1. Ablation studies

Recurrence versus reward engineering. Table 6 separates trajectory context (recurrence) from the reward-derived advantage weighting (final fold, 5 seeds). Recurrence yields a large, stable gain (+0.429 MCC; memoryless 0.493 → recurrent 0.922 at seed std 0.003). Adding advantage weighting on top does not reliably help: the mean falls only 0.059, but — more tellingly — the seed variance rises ∼40-fold (±0.003 → ±0.111) as one of five seeds collapses to 0.643. The gains of the sequence agents come from recurrence; reward engineering is neutral-to-harmful and destabilizing here.

Reward-weight sensitivity. Scaling four key reward coefficients by {0.5, 2.0} and retraining (Table 7) leaves the qualitative picture unchanged and exposes a clean dichotomy. The two imitation agents are *exactly invariant* — both Memoryless BC (0.493) and Recurrent BC (0.911) are unchanged across all eight perturbations — because neither uses the reward in its objective ($w_t = 1$). The reward-using agents instead vary, and the value-based agents remain unusable (IQL ≈0; CQL −0.09–0.31); the Decision Transformer is the most reward-sensitive, swinging over a 0.51 MCC range (0.411–0.920)—further evidence of its fragility. Agent behavior is thus not a reward-weight artifact, and the strongest agent (Recurrent BC) is the one the reward cannot perturb at all. Like the ablation and the selection search, this is a single-fold analysis (the final included fold, 5 seeds); its absolute MCCs therefore align with the single-fold selection (Table 9) rather than the multi-fold benchmark (Table 10), and we read it only for the qualitative invariance, not for effect sizes.

5.2. Supervised baselines: A binary-detection reference

Table 8 reports supervised classifiers on the identical leakage-free folds and 46-dimensional observation. On binary detection they are strong — RandomForest/XGBoost reach MCC ≈0.97, ROC-AUC ≈1.0 — and, as the benchmark below confirms, exceed every offline agent. We state this plainly and use it as a *binary-detection reference*, not a direct competitor: these baselines are per-flow binary detectors that quantify the detectable attack signal but do not output the three-tier triage action. Our aim is therefore not to outperform these detectors on binary detection but to assess how far offline policy learning recovers an operational triage policy—one that conditions on an alert's trajectory and is scored on operational quantities (expected return, containment, risk-alignment, action-correctness) rather than per-flow accuracy alone. We note that even a multi-class classifier recovers the three-tier action

Table 8
Supervised detection baselines (46-dim observation; 5 folds × 5 seeds). These provide a binary-detection reference, not a competing triage policy.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	MCC
Logistic Reg.	0.910 ± 0.006	0.999 ± 0.001	0.952 ± 0.003	0.999 ± 0.000	0.991 ± 0.005	0.944 ± 0.003
Random Forest	0.970 ± 0.012	0.975 ± 0.003	0.972 ± 0.005	0.999 ± 0.000	0.997 ± 0.002	$\mathbf{0.967 \pm 0.005}$
XGBoost	0.963 ± 0.015	0.979 ± 0.004	0.971 ± 0.006	1.000 ± 0.000	0.998 ± 0.001	0.965 ± 0.006

Table 9
Per-agent hyperparameter search on the UNSW-NB15 dataset (final fold, seed 42; single-fold *selection*, **bold** = selected). Whether these single-fold values persist is evaluated in Table 10.

Agent	Config	Val. MCC	Test MCC	Test F1
CQL	$\alpha = 1$	−0.042	−0.046	0.001
	$\alpha = 5$	**0.539**	**0.444**	**0.547**
	$\alpha = 10$	0.340	−0.011	0.018
	$\alpha = 20$	−0.042	−0.046	0.001
IQL	*all 9* (τ, β)	0.000	0.000	0.000
DT	$K = 10$	0.886	0.914	0.925
	$K = 25$	**0.905**	**0.924**	**0.937**
	$K = 50$	0.000	0.000	0.000

only moderately, predicting each flow independently (Section 5.4); the trajectory-aware agents instead add sequential modeling and operational scoring. The benchmark therefore asks how much of the detection signal a policy retains alongside these operational quantities.

5.3. Hyperparameter selection

Table 9 reports the per-agent search on the final validation fold (single seed) used to *select* each agent's configuration (CQL $\alpha = 5$, DT $K = 25$, IQL $\tau = 0.7, \beta = 3$); these selected configurations are then evaluated across all folds and seeds in the benchmark (Section 5.5). Selection is by validation MCC, with F1/ROC-AUC shown to expose degenerate policies. We flag that these are *single-fold* values: whether they persist is precisely the question the multi-fold benchmark answers, and — as shown there — they largely do not. IQL is degenerate at all nine (τ, β) settings, so no setting is a genuine optimum; we retain the paper default $\tau = 0.7, \beta = 3$.

5.4. Policy recovery

To gauge how learnable the three-tier policy is from the observation alone, we fit the same supervised classifiers to predict the behavior-policy *action* rather than the label over 5 folds × 5 seeds. Recovery is moderate: Cohen's κ ranges 0.38–0.43 (Logistic Regression 0.429, Random Forest 0.409, XGBoost 0.383), while accuracy (0.78–0.84) is inflated by the majority DISMISS class. Per class, DISMISS is recovered well (F1≈0.92), the rare INVESTIGATE and CONTAIN poorly (F1 ≤ 0.47 and ≤ 0.30). This features-only *reference point* partly reflects the class prior (CONTAIN is rare); the offline policy agents are assessed operationally in Table 11.

Recovered policy structure (illustrative: final fold, seed 42). Fig. 2 shows each agent's policy as P(action | attack category). The coarse analyst structure is recovered — benign NORMAL to DISMISS, most attacks to INVESTIGATE — but three failure modes appear. The value-based agents collapse to near-degenerate all-DISMISS: CQL and IQL route almost every category, attacks included, there. The Decision Transformer DISMISSES several attack categories (Exploits, Fuzzers, Worms), missing genuine attacks. And CONTAIN is used by two agents only: Recurrent BC and LSTM route high-severity *Exploits* there, partially recovering the analyst's containment rule; Recurrent BC also contains low-severity Analysis. The recurrent agents otherwise best separate benign from attack traffic; Memoryless BC over-INVESTIGATES.

Expert-action agreement (Final-fold, seed 42). This run can also be read as the expert-to-RL action confusion P(RL action | expert action), whose diagonal gives per-tier agreement with the reconstructed behavior policy. Frequency-weighted agreement is high for the recurrent and sequence agents (Recurrent BC 0.81, advantage-weighted GRU 0.79, DT 0.82, advantage-weighted LSTM 0.63, Memoryless BC 0.56), but unweighted per-class means are lower (0.50–0.59). The value-based agents show the gap most sharply: CQL and IQL reach 0.80 and 0.81 at chance-level per-class means (0.33), mapping every tier to DISMISS. The insight is that for most agents the three-tier action space collapses in practice to *two* tiers — the dominant DISMISS/INVESTIGATE contrast is recovered but CONTAIN is not — so agreement on the full three-tier policy is limited, consistent with the moderate supervised recovery of Section 5.4. Only Recurrent BC and LSTM exercise all three tiers, and they do so where it is warranted (containing high-severity Exploits). Because containment is nonetheless rare and imperfect, a natural deployment surfaces these CONTAIN recommendations to the analyst, who retains the final containment decision, while the learned policy carries the higher-volume DISMISS/INVESTIGATE triage. We stress that Fig. 2 is a single-run illustration (the held-out test split under the primary seed (42)) shown to expose behavior that scalar metrics hide; the quantitative verdict is the aggregated 5 × 5 detection and operational results of Tables 10–11, not this snapshot.

5.5. Main benchmark: The multi-fold verdict

Table 10 establishes the experimental verdict. The recurrent agents lead. Recurrent BC is the strongest agent, with the highest MCC (0.752 ± 0.147) and the best ROC-AUC (0.991) and PR-AUC (0.943) of any agent. The Decision Transformer follows in mean (0.713 ± 0.159) but is the most *unstable* across folds. The advantage-weighted GRU (0.666 ± 0.066) and LSTM (0.606 ± 0.032) are lower but more stable. Memoryless BC is stable but modest (0.473). The value-based agents collapse: CQL (0.078 ± 0.291) and IQL (0.008 ± 0.015) sit near zero, IQL retaining ROC-AUC 0.854 despite F1 0.009. Two contrasts visible here recur across datasets (Section 5.7): recurrence lifts MCC sharply over Memoryless BC (0.473 → 0.752/0.666), and removing reward-derived advantage weighting helps rather than hurts (Recurrent BC 0.752 > advantage-weighted GRU 0.666, a +0.086 gap).

Crucially, the single-fold selection of Table 9 does *not* survive: DT's single-fold 0.924 becomes 0.713 ± 0.159 and CQL's 0.444 becomes 0.078. A single held-out split would have concluded DT dominates and CQL recovers; leakage-free temporal cross-validation shows DT is high-variance and CQL collapses. This justifies both the selection procedure (single fold for tractable search) and the multi-fold protocol (for the reported verdict). Against the supervised reference (Table 8), the recurrent agents retain much of the ranking signal (ROC-AUC up to 0.991, at the baselines' ≈1.0), though on MCC (0.61–0.75) they remain below the ≈0.97 attained by the supervised baselines.

5.6. Operational verdict and reward–task misalignment

Table 11 completes the verdict with SOC-operational metrics. The recurrent agents produce coherent operational behavior — high action-correctness (≈0.83) and positive risk-alignment (up to 0.610) — while containing conservatively (rates ≤ 0.08). The reward objective is again misaligned with the task: CQL attains the *highest* expected return (71.57, versus 41.83 for Recurrent BC and 34.01 for DT) while its MCC is

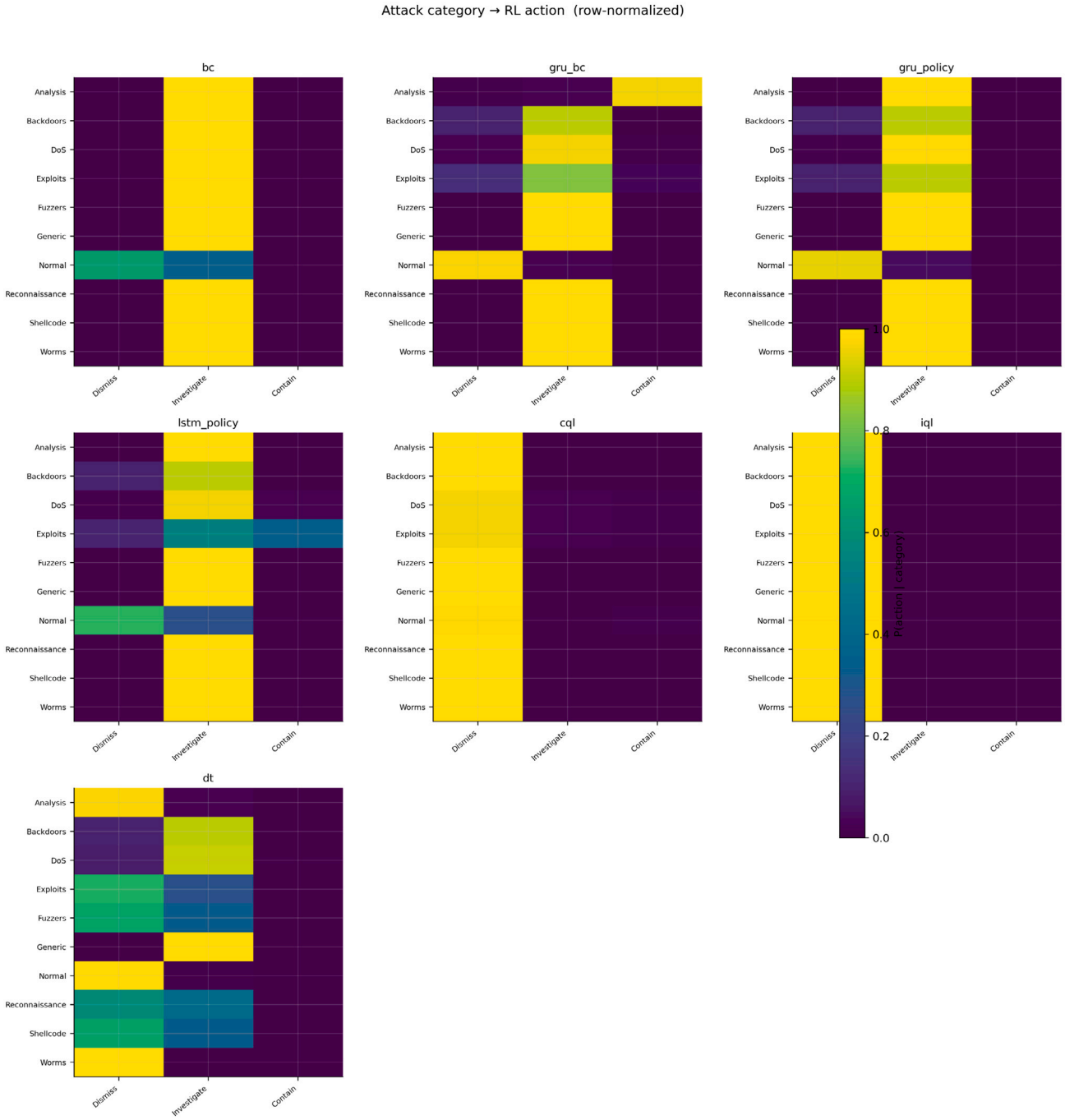


Fig. 2. Learned policy on the final fold and seed: per-category action distribution $P(\text{action} \mid \text{attack category})$ for each agent (row-normalized). Benign Normal maps to Dismiss and most attacks to Investigate; the value-based agents collapse to all-Dismiss, and the Contain tier is barely used. Single STAGE-2 final model per agent on the held-out test split (primary seed 42), shown for illustration only; aggregated scores are reported in Tables 10–11. Panels are labeled by agent key: `bc` (Memoryless BC), `gru_bc` (Recurrent BC, the strongest agent on both datasets), `gru_policy` and `lstm_policy` (advantage-weighted GRU/LSTM), `cql` and `iql` (value-based, which collapse to all-Dismiss), and `dt` (Decision Transformer).

near zero (0.078), it over-contains (0.156), and its risk-alignment is negligible (0.046)—it maximizes reward by over-containing, which is degenerate as triage. This independently corroborates the ablation: performance tracks recurrence, not the optimized reward. (Expected return is reported in the text rather than the table, as it is uninformative on CIC; Section 5.7.)

Computational cost. Table 12 reports average per-run wall-clock training time on UNSW-NB15; testing is negligible (≤ a few seconds). The Decision Transformer is the most expensive (≈106 s), about twice the recurrent agents (≈50 s); the ordering is consistent with CIC (Table 15), where DT is likewise the most expensive. Memoryless BC and CQL are cheapest (16–25 s). The strongest agent, Recurrent BC, is among the more economical (essentially the cost of the advantage-weighted GRU it outperforms), while the most expensive agent (DT) is the less reliable across datasets.

Table 10
Main benchmark on UNSW-NB15: across-fold mean ± std over 5 expanding-window folds (each a 5-seed mean; std = temporal sensitivity).

Agent	Precision	Recall	F1	ROC-AUC	PR-AUC	MCC
Recurrent BC	0.663 ± 0.168	0.995 ± 0.001	0.778 ± 0.129	0.991 ± 0.004	0.943 ± 0.027	**0.752 ± 0.147**
DT (return)	0.685 ± 0.200	0.888 ± 0.078	0.743 ± 0.151	0.916 ± 0.071	0.880 ± 0.088	0.713 ± 0.159
GRU (adv.)	0.541 ± 0.086	0.996 ± 0.002	0.695 ± 0.067	0.981 ± 0.003	0.848 ± 0.027	0.666 ± 0.066
LSTM (adv.)	0.477 ± 0.026	0.998 ± 0.000	0.640 ± 0.020	0.981 ± 0.003	0.853 ± 0.020	0.606 ± 0.032
Memoryless BC	0.350 ± 0.051	1.000 ± 0.000	0.517 ± 0.059	0.822 ± 0.025	0.352 ± 0.052	0.473 ± 0.048
CQL (value)	0.250 ± 0.161	0.392 ± 0.305	0.255 ± 0.209	0.555 ± 0.268	0.359 ± 0.182	0.078 ± 0.291
IQL (value)	0.121 ± 0.077	0.005 ± 0.005	0.009 ± 0.008	0.854 ± 0.025	0.424 ± 0.047	0.008 ± 0.015

Table 11
UNSW-NB15 operational metrics (5 folds × 5 seeds, mean ± std). MTTR and MTTC coincide and are reported as one column (MTTR/C). Expected return is included for completeness but is misaligned with triage quality: the highest returns accrue to the degenerate over-container CQL (71.57) and the advantage-weighted GRU/LSTM (64.82/58.67), while the best agent by MCC, Recurrent BC, is mid-pack (41.83). Columns match the CIC table (Table 14).

Agent	MTTD	MTTR/C	Contain rate	Correctness	Risk-align	Return
Recurrent BC	0.032 ± 0.016	0.640 ± 0.160	0.076 ± 0.065	0.796 ± 0.057	0.456 ± 0.172	41.83 ± 2.81
DT (return)	0.080 ± 0.108	0.052 ± 0.087	0.035 ± 0.077	0.833 ± 0.041	0.561 ± 0.154	34.01 ± 4.82
GRU (adv.)	0.017 ± 0.012	0.263 ± 0.300	0.030 ± 0.048	0.839 ± 0.027	**0.610 ± 0.061**	64.82 ± 11.31
LSTM (adv.)	0.011 ± 0.005	0.119 ± 0.139	0.019 ± 0.022	0.833 ± 0.034	0.542 ± 0.043	58.67 ± 7.48
Memoryless BC	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.837 ± 0.027	0.420 ± 0.043	36.99 ± 4.02
CQL (value)	1.025 ± 1.344	0.699 ± 0.716	0.156 ± 0.083	0.707 ± 0.094	0.046 ± 0.190	71.57 ± 17.54
IQL (value)	0.093 ± 0.179	0.175 ± 0.164	0.017 ± 0.015	0.838 ± 0.027	0.025 ± 0.026	25.75 ± 5.45

Table 12
Average per-run computational cost on UNSW-NB15 (wall-clock, mean over 5 folds × 5 seeds).

Agent	Training	Testing	Total	Relative cost
Recurrent BC	50 s (0.83 min)	2–3 s	52 s	Low
Memoryless BC	16 s (0.27 min)	2 s	18 s	Very low
GRU (adv.)	51 s (0.85 min)	2–3 s	53 s	Low
LSTM (adv.)	54 s (0.90 min)	2–3 s	57 s	Low
CQL (value)	25 s (0.42 min)	2 s	27 s	Very low
DT (return)	106 s (1.77 min)	4 s	110 s	Moderate
IQL (value)	55 s (0.92 min)	2 s	57 s	Low

5.7. Generalization to CIC-IDS-2017

We repeat the benchmark on CIC-IDS-2017 with the identical protocol and the seven agents (Table 13). Because CIC's attacks are concentrated on specific days, only two expanding-window folds meet the ≥1.5% train attack-rate inclusion criterion; the earlier, attack-sparse folds are excluded. Results are therefore across 2 folds × 5 seeds — a weaker temporal-sensitivity estimate than UNSW's five folds — and we read them accordingly.

Three conclusions replicate on this independent dataset. First, recurrence helps: Recurrent BC (0.621) exceeds Memoryless BC (0.492) by 0.129. Second, reward-derived advantage weighting hurts: Recurrent BC ($w_t = 1$, 0.621) exceeds the advantage-weighted GRU (0.526) by 0.095—a benchmark-level confirmation of the single-fold ablation (Section 5.1) on a second dataset. Third, the value-based agents collapse (CQL −0.025, IQL −0.053). One finding diverges: the Decision Transformer, high-variance but non-trivial on UNSW, collapses entirely on CIC (MCC 0.000, ROC-AUC 0.525; a degenerate always-escalate policy), reinforcing that offline RL is brittle across datasets. Operationally (Table 14), Recurrent BC is both the strongest detector and the most risk-aligned agent (0.567), while the value-based agents again over-contain (CQL containment rate 0.667) with negligible or negative risk-alignment. Expected return is omitted as uninformative (all agents ≈100 with large across-fold variance), so operational degeneracy is read from containment rate and risk-alignment; MTTR and MTTC coincide for every agent and are shown as one column.

Computational cost. Table 15 reports average per-run wall-clock time on CIC-IDS-2017; testing is negligible for every agent (≤ 13 s). Training cost spans two orders of magnitude. The Decision Transformer is the most expensive (≈170 s), about 1.4× the recurrent policies (≈118–123 s) and 10× Memoryless BC, yet it collapses on CIC (Table 13), the worst cost–benefit in the benchmark. Memoryless BC and the value-based agents are cheapest (21–62 s), but the value-based agents also collapse. Notably, Recurrent BC (168 s) is both more accurate and *cheaper* than the advantage-weighted GRU (189 s): the return-based advantage weighting adds computation (return and advantage estimation) while reducing accuracy, so Recurrent BC dominates the advantage-weighted policy on both axes. The strongest agent is thus among the more economical, while the most expensive agent is degenerate.

5.8. Statistical significance

We assess agent differences using three non-parametric tests on per-seed detection MCC. The Friedman omnibus test rejects the null hypothesis of equal agent performance on both datasets (UNSW-NB15: $\chi^2(6) = 26.9$, $p = 1.5 \times 10^{-4}$; CIC-IDS-2017: $\chi^2(6) = 27.8$, $p = 1.0 \times 10^{-4}$), with Recurrent BC achieving the best (lowest) mean rank on both datasets (1.4 on UNSW-NB15 and 1.0 on CIC-IDS-2017). Nemenyi post-hoc comparisons identify Recurrent BC as significantly better than several value-based agents (e.g., on UNSW-NB15: versus CQL, $p<10^{-4}$, and versus IQL, $p = 4.4\times10^{-4}$; on CIC-IDS-2017: versus IQL, $p = 1.4\times10^{-4}$, CQL, $p = 4.4 \times 10^{-4}$, and DT, $p = 2.5 \times 10^{-4}$). Because the per-fold signed-rank test is limited by the five-seed sample size ($p_{\min} = 0.0625$), we additionally report pooled Wilcoxon signed-rank tests across all (fold, seed) pairs for the two ablation comparisons. The recurrence effect is significant on both datasets: Recurrent BC outperforms Memoryless BC with $p = 1.8 \times 10^{-7}$ (24/25 paired wins) on UNSW-NB15 and $p = 0.002$ (10/10) on CIC-IDS-2017. The reward-engineering effect is dataset-dependent: Recurrent BC significantly outperforms the advantage-weighted GRU on CIC-IDS-2017 ($p = 0.002$, 10/10), whereas the difference is not significant on UNSW-NB15 ($p = 0.06$, 14/25), with a median difference of approximately zero. The direction of the paired differences also varies across folds, consistent with the ablation results showing no consistent performance benefit from advantage weighting.

5.9. Discussion

The evaluation protocol changes the conclusion. Single-fold, DT looks dominant (0.924) and CQL recovered (0.444); under leakage-free temporal CV, DT is high-variance (0.713 ± 0.159) and CQL collapses

Table 13
Generalization to CIC-IDS-2017: across-fold mean ± std over the 2 included expanding-window folds (each a 5-seed mean). Only two folds meet the ≥ 1.5% attack-rate inclusion criterion, so the across-fold std is a 2-point estimate.

Agent	Precision	Recall	F1	ROC-AUC	PR-AUC	MCC
Recurrent BC	0.659 ± 0.020	0.922 ± 0.034	0.768 ± 0.002	0.846 ± 0.093	0.721 ± 0.155	**0.621 ± 0.012**
GRU (adv.)	0.548 ± 0.010	0.994 ± 0.007	0.706 ± 0.007	0.834 ± 0.057	0.681 ± 0.091	0.526 ± 0.030
Memoryless BC	0.591 ± 0.049	0.818 ± 0.048	0.677 ± 0.062	0.828 ± 0.090	0.687 ± 0.148	0.492 ± 0.075
LSTM (adv.)	0.500 ± 0.008	0.996 ± 0.005	0.665 ± 0.008	0.816 ± 0.054	0.632 ± 0.035	0.451 ± 0.004
DT (return)	0.367 ± 0.016	1.000 ± 0.000	0.537 ± 0.017	0.525 ± 0.156	0.437 ± 0.043	0.000 ± 0.000
CQL (value)	0.342 ± 0.029	0.647 ± 0.330	0.368 ± 0.186	0.551 ± 0.076	0.404 ± 0.037	−0.025 ± 0.049
IQL (value)	0.259 ± 0.121	0.460 ± 0.300	0.313 ± 0.197	0.422 ± 0.214	0.448 ± 0.108	−0.053 ± 0.050

Table 14
CIC-IDS-2017 operational metrics (2 folds × 5 seeds, mean ± std). MTTR and MTTC coincide for every agent and are reported as one column (MTTR/C). Expected return is shown but is uninformative on CIC: every agent clusters near ∼100 regardless of triage quality (Recurrent BC 101.1 vs. the degenerate IQL 88.4), being dominated by the two folds' differing reward scales—reinforcing that return is misaligned with useful triage.

Agent	MTTD	MTTR/C	Contain rate	Correctness	Risk-align	Return
Recurrent BC	0.286 ± 0.059	29.05 ± 1.19	0.093 ± 0.077	0.635 ± 0.019	**0.567 ± 0.017**	101.1 ± 57.9
GRU (adv.)	0.016 ± 0.022	31.26 ± 3.04	0.175 ± 0.076	0.609 ± 0.022	0.255 ± 0.044	100.2 ± 60.2
Memoryless BC	1.167 ± 0.272	30.47 ± 3.71	0.300 ± 0.401	0.654 ± 0.049	0.457 ± 0.077	102.3 ± 57.1
LSTM (adv.)	0.029 ± 0.041	42.36 ± 3.66	0.072 ± 0.043	0.621 ± 0.014	0.327 ± 0.008	100.1 ± 60.1
DT (return)	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.632 ± 0.016	−0.002 ± 0.003	99.9 ± 59.7
CQL (value)	2.018 ± 0.783	2.80 ± 2.19	0.667 ± 0.367	0.506 ± 0.026	0.019 ± 0.112	102.9 ± 54.1
IQL (value)	3.903 ± 0.329	5.39 ± 1.58	0.493 ± 0.261	0.478 ± 0.059	−0.076 ± 0.003	88.4 ± 76.9

Table 15
Average per-run computational cost on CIC-IDS-2017 (wall-clock). Testing time is negligible for all agents.

Agent	Training	Testing	Total	Relative cost
Recurrent BC	116 s (1.93 min)	2 s	118 s	Low
Memoryless BC	15 s (0.26 min)	2 s	17 s	Very low
GRU (adv.)	121 s (2.01 min)	2 s	123 s	Low
LSTM (adv.)	116 s (1.93 min)	2 s	118 s	Low
CQL (value)	23 s (0.38 min)	2 s	25 s	Very low
DT (return)	167 s (2.78 min)	3 s	170 s	Moderate
IQL (value)	51 s (0.86 min)	2 s	53 s	Low

(0.078), while the strongest agent — on both datasets — is Recurrent BC (0.752 on UNSW). This divergence — from the single-fold selection (Table 9) to the multi-fold verdict (Table 10) — is itself a central result and motivates the protocol: a single held-out split can produce a substantially different ranking from a temporally rigorous evaluation.

Detection is a reference point, not the task. Tuned supervised baselines exceed all agents on binary detection (≈0.97 vs. ≈0.75), providing a strong empirical reference for the detection component of the task. The trajectory-aware agents nevertheless retain substantial ranking signal (ROC-AUC near the supervised baselines, although MCC is lower) while producing a sequential triage policy evaluated using operational quantities (risk-alignment, containment rate, action-correctness) that per-flow binary detection does not address. The goal is therefore to assess how far offline policy learning recovers an operational triage policy while retaining useful detection performance, rather than to outperform supervised detectors on binary detection. We also find that the value-based agents do not yield a consistently usable policy and that containment recovery remains difficult.

The reward objective is misaligned with the task. The reward-maximizing agent (CQL, return 71.57) has near-zero MCC (0.078) and over-contains; the ablation independently shows that reward-derived advantage weighting does not reliably improve performance and can destabilize it (−0.059 mean, ∼40× the seed variance). These results suggest that the observed gains are more closely associated with recurrence than with reward engineering alone.

Offline value-based RL fails robustly, and the failures are metric-masked. IQL and CQL collapse across folds, across all nine IQL settings, and across reward-weight perturbations. Because these failures can present as single-action or over-containing policies despite relatively high ROC-AUC or return, threshold-based metrics (MCC, F1), rather than ROC-AUC or return alone, are needed to characterize them—a methodological observation that may be relevant beyond this dataset.

Cross-dataset generalization. The central conclusions replicate on CIC-IDS-2017 (Table 13): recurrence helps, reward-derived advantage weighting hurts, and the value-based agents collapse. On both datasets recurrence lifts MCC over Memoryless BC, and Recurrent BC ($w_t = 1$) exceeds the advantage-weighted GRU (UNSW 0.752 > 0.666; CIC 0.621 > 0.526). Recurrent BC is the strongest agent on *both* datasets. The Decision Transformer diverges: it is second on UNSW but the most variable (0.713 ± 0.159) and collapses entirely on CIC (0.000), indicating that its UNSW performance does not transfer under the evaluated CIC protocol. One limitation is the fold count: CIC yields only two attack-dense folds under the inclusion criterion, so its temporal-sensitivity estimate is less extensive than UNSW's five-fold estimate.

Deployment. We distinguish the strongest agent from the policy selected for deployment. Recurrent BC is the strongest benchmark policy and obtains the best MCC on both datasets; however, its across-fold variance on UNSW is high (±0.147), whereas the advantage-weighted GRU attains nearly the same DISMISS/INVESTIGATE behavior (Fig. 2) at roughly one-quarter of that variance (±0.066). For an operational SOC, where consistency may be preferable to peak accuracy, we therefore favor the lower-variance GRU as the deployment candidate, with Recurrent BC retained as the best-performing benchmark agent and the human analyst retaining the CONTAIN decision.

High-severity safeguard. A deployed triage policy should not silently drop severe alerts. We therefore compose the learned GRU policy with a deterministic *severity floor*: if the policy would DISMISS a flow whose estimated severity is medium or high, the action is raised to INVESTIGATE, while CONTAIN remains analyst-owned. The floor is motivated by an audit on the held-out split (primary seed): without it, GRU auto-DISMISSES 10.0% of high-severity and 0.3% of medium-severity alerts (the high-severity rate is dominated by Exploits, $n = 3643$), comparable to the other recurrent agents (10%–14%) but far below the Decision Transformer (72%) and the value-based agents (≥96%), which do not satisfy this safeguard criterion. With the floor, no medium- or high-severity alert can be auto-dismissed, by construction, when the ground-truth severity category is available. Two limitations apply. First, in this

offline evaluation the severity signal is the ground-truth category, so the floor is exact; in deployment it would key on a *predicted* severity signal (e.g. the supervised detector of Table 8), and the guarantee is therefore bounded by that estimate's reliability. Second, the floor trades additional analyst review volume for this safety property, since it converts residual DISMISS decisions on severe traffic into INVESTIGATE. To evaluate the safeguard without using the ground-truth category at inference, we additionally validate a *predicted*-severity floor built only from the leakage-free observation features: a classifier is trained on the final fold's training window (1,700,685 transitions), its escalation threshold is selected on the validation window to target 99% recall on medium/high-severity flows, and it is evaluated on the held-out test window (360,227 transitions). In isolation this predicted floor escalates 99.95% of held-out high-severity attacks (Exploits: 99.95%, $n = 3643$; all high-severity $n = 3663$) and 98.90% of medium/high attacks, at a review-volume cost of 3.75% of flows. Composed with the policy, the end-to-end system — the policy followed by the predicted-severity floor — auto-dismisses zero high-severity and zero medium/high attacks (recall 100.00% on both, $n = 3663$ and 8793). The policy alone would auto-dismiss 367 high-severity attacks (10.0%), and the label-free floor escalates each of these cases. Across all 64,613 held-out attacks, the deployed system auto-dismisses only 2 (0.003%; recall 99.997%), both low-severity. Thus, the offline audit provides a measured evaluation of a label-free safeguard rather than relying solely on a ground-truth-dependent guarantee. The only lower-volume category showing an isolated-floor weakness is Analysis (88.3%, $n = 77$), which the policy's own escalations nonetheless cover end-to-end. These figures are tracked in shadow mode. Accordingly, deployment implementation applies this *predicted*-severity floor — a classifier over observable features, trained on the training window and thresholded on validation — as the operative safeguard; the ground-truth category is used only for the offline reference audit and is never required at inference. Rare categories (Backdoors, Shellcode, Worms) are too sparse on a single split to estimate reliably and are not reported individually.

Deployment evaluation and shadow mode. The interactive dashboard and the quantitative evaluation are now computed on the same data. The console replays the complete final-fold held-out test partition — 381,008 flows, reconstructed to 360,227 transitions — in a single chronological pass, and its reported detection, operational, and severity-audit figures reproduce the offline values for the deployed policy on that fold (Section 6.1). All reported detection, operational, and severity-audit results are computed on the *full* held-out test partitions under leakage-free temporal cross-validation—381,008 flows on the final fold, and up to 2.1 million on the earliest, across 5 folds × 5 seeds (Tables 10–11). The deployed policy is therefore evaluated on hundreds of thousands of held-out flows per fold. That said, this evaluation remains *retrospective*: it scores the policy against a label-derived analyst heuristic on historical traffic, and the adjudication recorded in the console is a surrogate for, rather than a record of, analyst decisions.

Genuine live-readiness would require a *shadow-mode* deployment — running the policy in parallel with human analysts, without acting on its outputs, over weeks to months of production traffic — and measuring (i) agreement with analyst dispositions, (ii) the false-dismiss rate on medium/high-severity alerts (the safety metric of the severity floor), (iii) the additional review volume the floor induces, and (iv) stability under concept drift, with periodic retraining. We therefore present offline evaluation on full held-out traffic as a necessary precondition, while recognizing shadow-mode evaluation as the next step before operational use.

6. Deployment considerations

The trained GRU policy is deployed in an advisory, human-in-the-loop configuration (Fig. 3) that reflects how modern security operations centers consume AI-assisted recommendations without ceding authority over disruptive mitigation actions. During deployment, raw network flows are ingested and transformed into a streaming test feed through feature extraction and normalization using training-period statistics only, thereby preserving the leakage-free discipline established during offline evaluation. The processed flows are streamed sequentially in a single pass to emulate realistic live traffic conditions.

1. **Data stream ingestion and feature extraction.** Real-time network traffic and flow records are continuously preprocessed into numerical observation vectors. Temporal ordering, feature normalization, and pseudo-trajectory generation ensure consistency with the offline protocol while enabling realistic streaming inference.
2. **Policy loading and validation.** The trained GRU policy is loaded from the deployment repository together with its configuration and provenance metadata; a validation stage performs dimensionality and inference smoke testing.
3. **Threat scoring and action recommendation.** Each incoming flow is evaluated by the `ModelServer`, which computes a threat score, a confidence estimate, and one of three tiered SOC responses (DISMISS, INVESTIGATE, CONTAIN). A safety gate evaluates policy confidence and operational risk according to the configured deployment mode (SHADOW, ADVISORY, AUTONOMOUS).
4. **Human-gated decision execution.** Recommendations that reach escalation tiers are routed into a bounded, newest-first analyst review queue. Analysts may approve, override, or escalate, while containment actions remain under explicit human authority.
5. **Live monitoring and dashboard visualization.** Detection metrics and SOC key-performance indicators are computed continuously, including rolling F1, ROC-AUC, MCC, confusion matrices, action distributions, and attack-category breakdowns. A live drift monitor computes the Jensen–Shannon divergence between the live action distribution and the training behavior prior.
6. **Feedback logging and continuous lifecycle monitoring.** Every analyst decision is logged with the model recommendation, executed action, threat score, timestamp, and ground-truth label where available, and exported into a labeled retraining dataset that closes the deployment feedback loop.

This deployment architecture intentionally separates recommendation from authority: the policy accelerates high-volume triage decisions, while disruptive containment remains a human responsibility. The advisory posture is thus not only a safety mechanism but a faithful operational reflection of the policy's learned behavior, which consistently prioritizes accurate alert graduation between dismissal and investigation while reserving containment for analyst judgment.

6.1. Evaluation of the deployed dashboard

Figs. 4–9 show the deployed operational console executing a full-feed replay of the final fold's held-out test partition: the complete 381,008-flow test block, which reconstructs to 360,227 pseudo-trajectory transitions (Table 5), streamed in a single chronological pass. The console runs in *shadow* mode with the deployed advantage-weighted GRU policy on the PyTorch backend: the policy scores every event and issues a recommendation, while every escalation is resolved by the analyst stage rather than acted upon automatically. The reconciliation counter records complete coverage with no dropped or duplicated events—360,227 flows scored, 360,227 agent decisions, and a dataset of 360,227 transitions. This replay replaces the small stratified sample reported previously, so the evaluation below is computed on the same held-out transitions as the offline results of Section 5 but over two orders of magnitude more events; it is summarized end-to-end in Table 16.

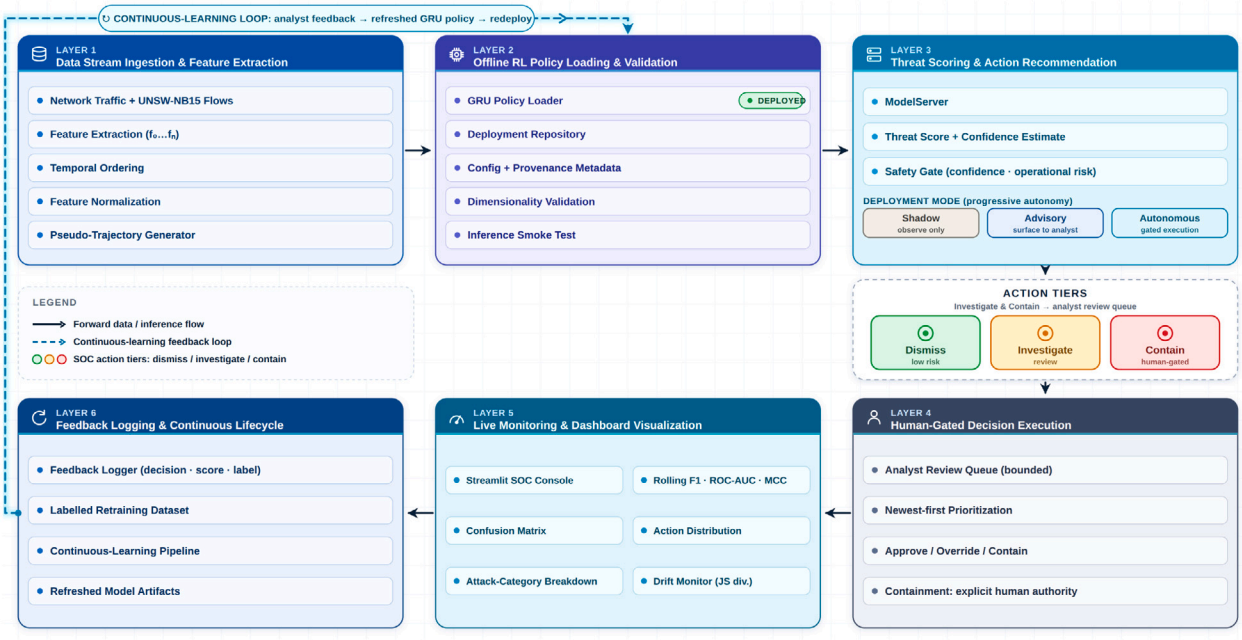


Fig. 3. End-to-end architecture of the SOC-RL threat-triage platform, a human-in-the-loop system deploying an offline-trained GRU policy. The pipeline forms a closed loop of six stages: ingestion and pseudo-trajectory construction; policy loading and validation; threat scoring and tiered action recommendation via the `ModelServer`, safety-gated by deployment mode; human-gated execution through a bounded analyst review queue; live monitoring of F1, ROC-AUC, MCC, confusion matrices, and a Jensen–Shannon drift monitor; and feedback logging and continuous-learning retraining. Solid arrows denote forward inference flow; the dashed arrow denotes the continuous-learning feedback loop.

Table 16
Full-feed deployment console session on the final-fold held-out test partition (360,227 reconstructed transitions from 381,008 flows; deployed advantage-weighted GRU policy, shadow mode). All quantities are read from the session report of Fig. 9 and reconcile with the panels of Figs. 4–8.

Quantity	Value
Stream	
Transitions scored	360,227
Attack/benign transitions	64,613/295,614
Raw policy action tiers	
Dismiss	281,695 (78.2%)
Investigate	78,531 (21.8%)
Contain	1 (<0.1%)
Detection (binary mapping)	
F1/ROC-AUC/PR-AUC	0.8971/0.9815/0.8402
MCC	0.8783
Precision/recall	0.8176/0.9937
TP/FP/TN/FN	64,207/14,325/281,289/406
Accuracy/FPR	0.9591/0.0485
Severity floor (shadow-mode audit)	
Raw auto-dismiss, high severity ($n = 3663$)	10.0%
Raw auto-dismiss, medium severity ($n = 5130$)	0.3%
Floor overrides (review-volume cost)	383 (+0.1%)
Post-floor medium/high auto-dismiss	0.0%
Residual auto-dismissed attacks (all low severity)	23 (0.04%)
Analyst adjudication	
Review queue (agent-escalated + floor-rescued)	78,915 (78,532+383)
Contained/approved/overridden	4577/60,013/14,325
Uphold rate	81.8%
Post-intervention tiers	
Dismiss/investigate/contain	295,639/60,011/4577
Policy drift vs. training prior (JS divergence)	0.0403

Raw policy behavior and detection performance. Before any human adjudication (Fig. 4), the policy dismisses 281,695 transitions (78.2%), routes 78,531 (21.8%) to *Investigate*, and issues a single autonomous *Contain* recommendation, yielding an alert rate of 21.8%. Scored against the ground-truth labels mapped to the three-tier scheme, it attains an F1 of 0.897, a ROC-AUC of 0.982, an MCC of 0.878, a precision of 0.818, a recall of 0.994, and a PR-AUC of 0.840. These in-situ values are consistent with the offline multi-seed estimate for the same agent on this fold (advantage-weighted recurrent policy, MCC 0.864 ± 0.111, F1 0.882 ± 0.100; Table 6), indicating that the deployment pipeline — feature extraction, training-period normalization, policy loading, and streaming inference — preserves behavior consistent with the evaluated policy. The complementary panels (Fig. 5) resolve this aggregate behavior temporally and per category through the threat timeline, the streaming F1 trace, the rolling confusion matrix, and the attack-category-to-action mapping, so that recommendations remain attributable to specific attack families rather than to summary statistics alone.

Binary detection view. For detection scoring, the three-tier output collapses to a binary problem: *Dismiss* is the negative (benign) class, whereas *Investigate* and *Contain* jointly form the positive class, since both flag traffic for review. Under this mapping the full stream yields TP = 64,207, FP = 14,325, TN = 281,289, and FN = 406, an accuracy of 95.9%, and a false-positive rate of FPR = 14,325/295,614 = 4.85%. The 64,613 attack-labeled and 295,614 benign transitions partition the stream exactly, and the per-category coverage table (Fig. 8) accounts for all of them. At this scale, the perfect recall observed on a small stratified sample does not persist: 406 attacks (0.6% of all attacks) are dismissed by the raw policy. This residual false-dismiss mass is the primary target of the severity floor, and it is therefore the floor — rather than the policy's raw recall — that provides the safety constraint evaluated here.

Severity floor under full-feed load. The shadow-mode panel of Fig. 4 audits the floor on the full stream. Without it, the raw policy auto-dismisses 10.0% of high-severity alerts ($n = 3663$, dominated by Exploits, $n = 3643$) and 0.3% of medium-severity alerts ($n = 5130$), reproducing on the identical partition the offline audit reported in

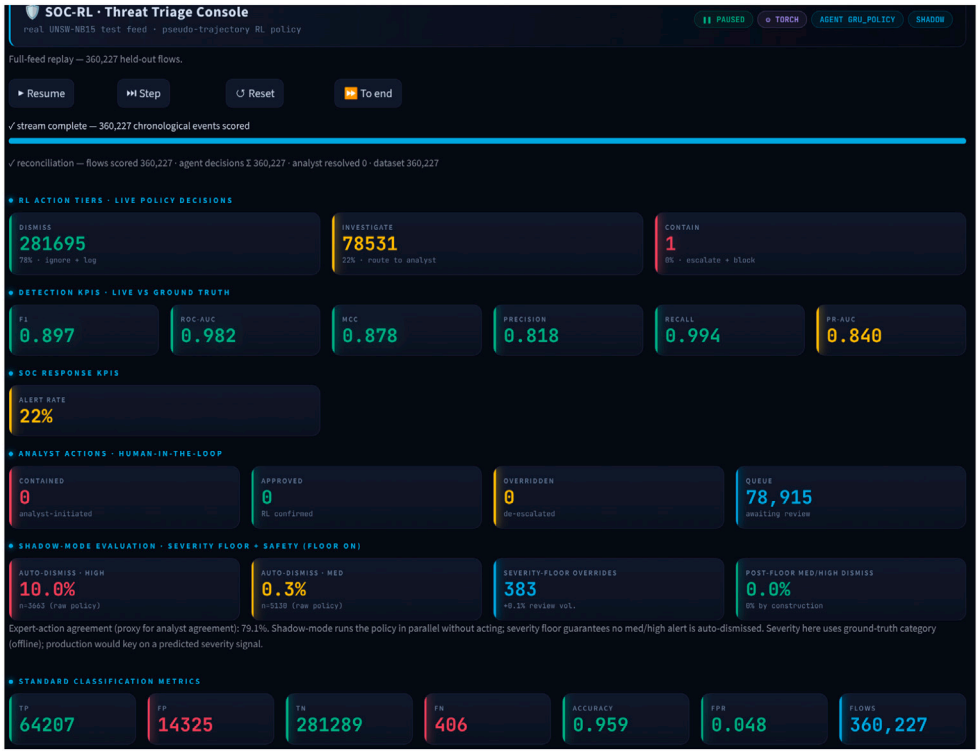


Fig. 4. Deployed SOC-RL console at completion of the full-feed replay, before human adjudication. The reconciliation line confirms that all 360,227 held-out transitions were scored. The panels report the raw RL action tiers, live detection KPIs against ground truth (F1, ROC-AUC, MCC, precision, recall, PR-AUC), the SOC alert rate, the empty analyst-action counters with the full 78,915-item queue awaiting review, the shadow-mode severity-floor audit, and the standard classification counts.

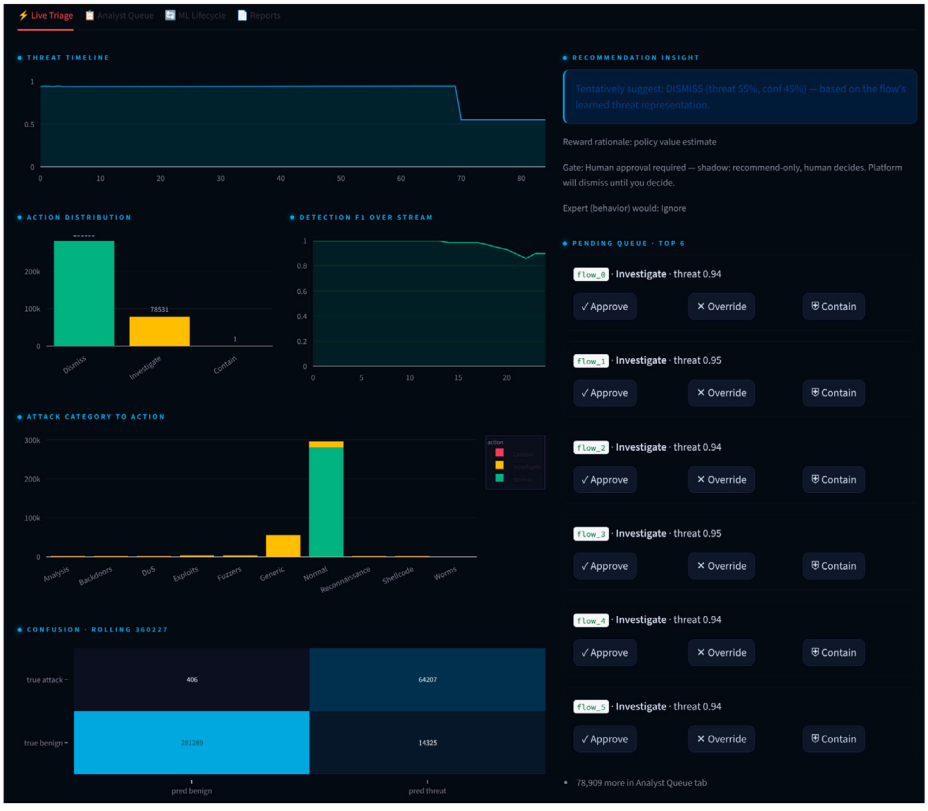


Fig. 5. Complementary live-triage panels for the same session: threat timeline, action distribution over the full stream, streaming F1, attack-category-to-action mapping, the rolling confusion matrix over all 360,227 transitions, and the recommendation-insight and pending-queue views through which individual alerts are surfaced for adjudication.

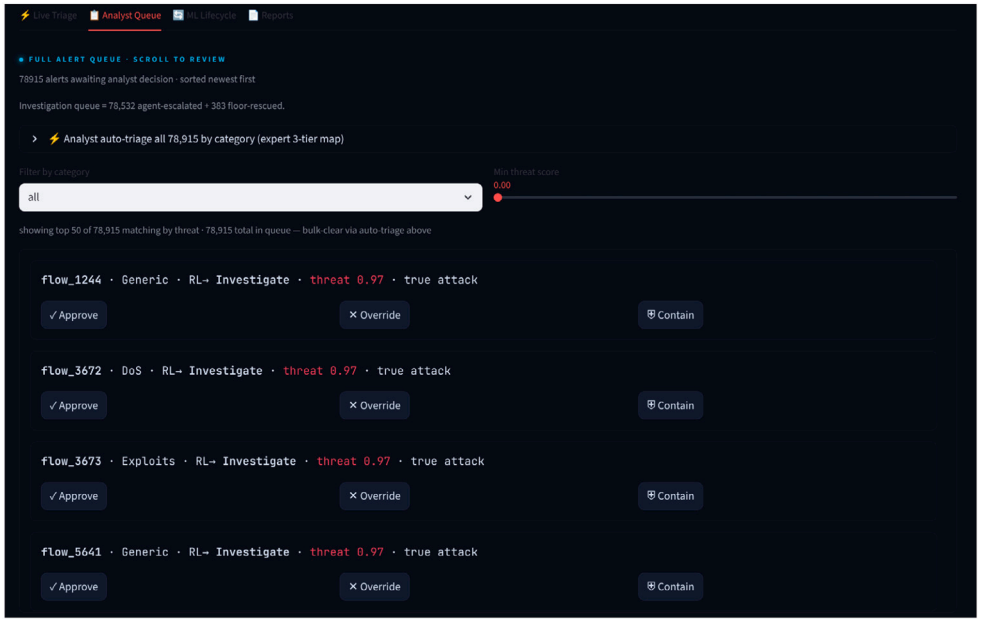


Fig. 6. Analyst review queue: 78,915 alerts awaiting human adjudication, composed of 78,532 agent-escalated alerts and 383 rescued by the severity floor. Each entry exposes the flow identifier, attack category, the policy's recommended tier, its threat score, and the ground-truth outcome, and is resolved by *approve* (confirm investigation), *override* (dismiss as benign), or *contain* (block or escalate).

Section 5.9. The floor rescues 383 such alerts, raising each from *Dismiss* to *Investigate*, after which the post-floor medium/high auto-dismiss rate is 0.0% by construction. The cost of this constraint is a review-volume increase of 383 alerts, or 0.1% of the stream. Because the 383 rescues are drawn entirely from the 406 raw false dismissals, only 23 attacks (0.04% of all attacks) remain auto-dismissed at the end of the pipeline, and all 23 are low severity by construction of the floor. Two caveats carry over from Section 5.9. First, the console's offline floor keys on the ground-truth category, so the constraint is exact for this evaluation; the deployment implementation instead keys on a predicted-severity signal estimated from observable features only, and the corresponding operational guarantee is therefore bounded by the reliability of that estimate. Second, the console reports an expert-action agreement of 79.1% as a proxy for analyst agreement, which is an offline surrogate rather than a measurement against real analyst dispositions.

Human-in-the-loop adjudication. The escalated alerts enter the bounded, newest-first analyst review queue of Fig. 6, which holds 78,915 items: the 78,532 agent-escalated alerts plus the 383 rescued by the severity floor. Adjudication of the full queue yields 4577 containments, 60,013 approvals, and 14,325 overrides, for an uphold rate of 81.8%. Two identities make the human stage auditable against the automated evaluation. The 14,325 overrides coincide exactly with the 14,325 false positives of the binary confusion matrix, indicating that human judgment removes the false alarms raised by the policy. Likewise, the 64,590 upheld alerts decompose exactly as 64,207 true positives plus 383 floor-rescued attacks, showing that every upheld alert corresponds to an attack under the ground-truth labels. The queue drains to zero, and all 78,915 decisions are captured as labeled feedback.

Post-intervention state and lifecycle. Folding the adjudications back into the stream — overrides rejoin the benign class, approvals remain under investigation, and containments are analyst initiated — yields the post-intervention three-tier distribution of Fig. 7: 295,639 *dismiss* (82.1%), 60,011 *investigate* (16.7%), and 4577 *contain* (1.3%). Containment is therefore exercised on 1.3% of the stream and is, by design, entirely analyst initiated: the policy itself recommended containment once in 360,227 decisions, consistent with the conservative behavior observed offline (Fig. 2). The live drift monitor reports a Jensen–Shannon divergence of 0.0403 between the deployed and training-period action distributions, indicating limited divergence over the full evaluation window while remaining larger than a short sample would suggest, since the full feed spans the complete evaluation period rather than a stratified slice. The ML-lifecycle panel (Fig. 8) records model provenance, per-category coverage across all ten classes, and the export of the 78,915 adjudicated decisions as a labeled retraining set, closing the feedback loop; the consolidated session report (Fig. 9) provides an auditable, per-session account of provenance, performance, drift, and analyst activity.

Operational reading. On the full held-out partition, the deployed system auto-clears 78.2% of traffic and concentrates human effort on the 21.8% escalation minority, of which 81.8% is upheld; the residual review overhead imposed by the safety floor is 0.1% of the stream. Analyst effort is spent on 78,915 of 360,227 events, and the errors surfaced to analysts remain conservative by construction: false alarms are intercepted before any disruptive action, at a false-escalation rate of 18.2% of escalations, or 4.0% of the full stream. We are explicit about what this evaluation does and does not establish. It establishes that the deployed artifact exhibits performance consistent with the offline evaluation on the complete held-out partition, that the severity floor operates as intended at scale, and that the human-gated loop reconciles with the automated evaluation. It does not establish live readiness: the adjudication is a label-derived surrogate for analyst behavior rather than a record of analyst decisions, and the shadow-mode study described in Section 5.9 over weeks to months of production traffic remains the required next step.

7. Threats to validity and mitigation strategies

Construct validity. Decision trajectories are reconstructed from flow records rather than observed from analyst sessions, and rewards are modeled from a multi-objective cost function rather than measured from operational outcomes. Our results therefore characterize policy recovery under a principled reward specification, not the replay of ground-truth analyst behavior; we mitigate arbitrariness by fixing the reconstruction rules and reward weights in advance and applying them identically across datasets.

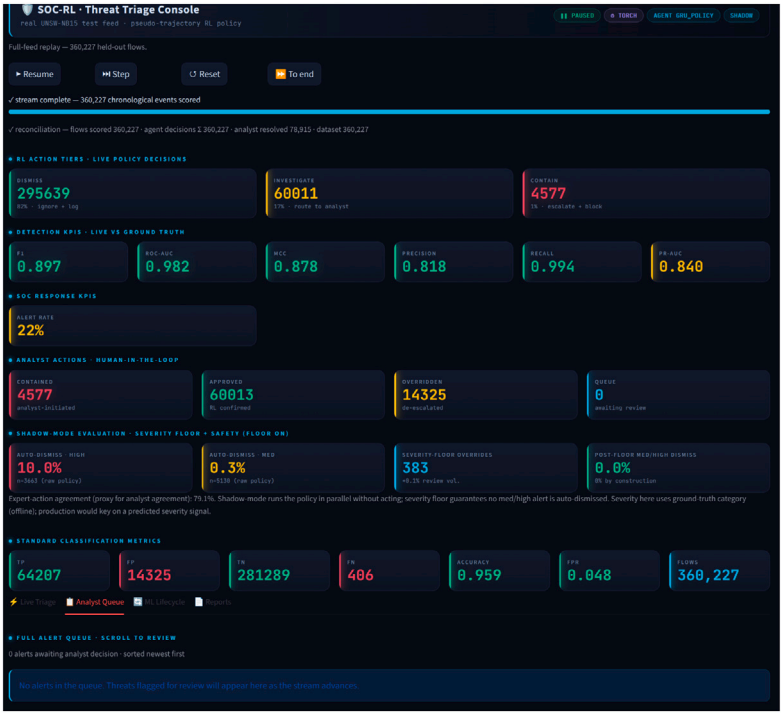


Fig. 7. Operational view following human-analyst intervention on the full feed. With the queue drained to zero, the tier distribution settles at 295,639 *dismiss*, 60,011 *investigate*, and 4577 *contain*; the analyst-action counters record 4577 containments, 60,013 approvals, and 14,325 overrides. Detection KPIs and classification counts are unchanged, since they score the policy's recommendations rather than the adjudicated outcome, so the panel shows directly that the 14,325 overrides coincide with the 14,325 false positives.

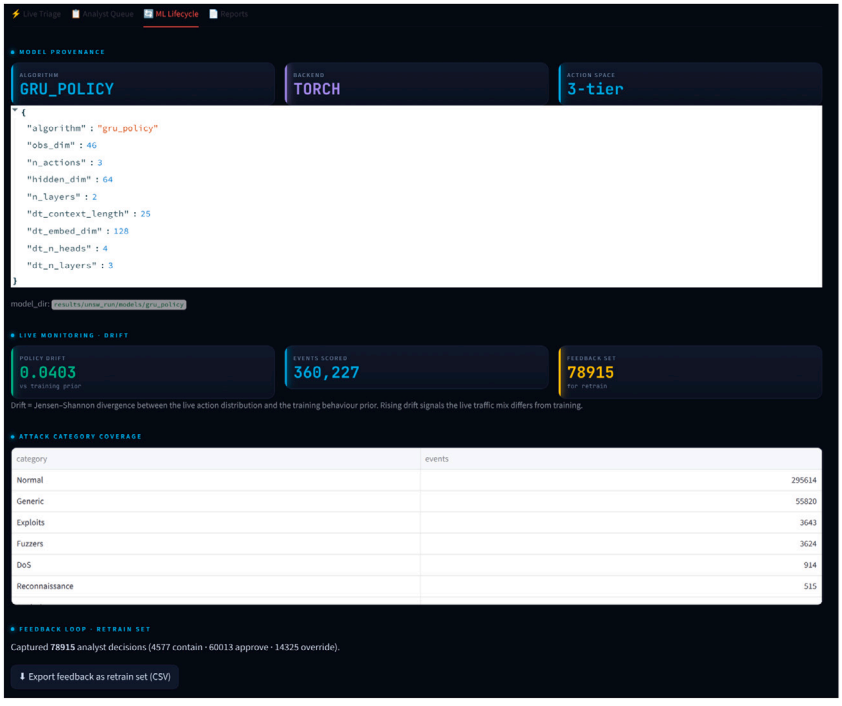


Fig. 8. ML-lifecycle panel: model provenance for the deployed GRU policy (algorithm, observation dimension, action space, architecture, model directory), live drift monitoring (Jensen–Shannon divergence 0.0403 against the training behavior prior over 360,227 scored events), attack-category coverage over the full stream, and the export of 78,915 adjudicated analyst decisions as a labeled retraining set that closes the deployment feedback loop.

Internal validity. The behavior policy is derived from dataset labels, so agents are evaluated on how well they recover label-consistent triage rather than on novel-attack discovery. This is a deliberate scoping choice, and our leakage-free protocol removes the most common confound — temporal leakage — that would otherwise inflate this recovery. The uniform, performance-independent fold-inclusion criterion further ensures that agents are never credited for folds whose training window cannot support learning.



SOC-RL · Triage Session Report

Generated 2026-08-11 01:19:10 · agent GRU_POLICY backend torch mode shadow · 360,227 / 360,227 events scored

MODEL PROVENANCE

algorithm	gru_policy
obs_dim	46
n_actions	3
hidden_dim	64
n_layers	2
dt_context_length	25
model_dir	results/unsw_run/models/gru_policy

RL ACTION TIERS

Dismiss	295639 (82.1%)
Investigate	60011 (16.7%)
Contain	4577 (1.3%)

DETECTION PERFORMANCE

F1	0.8971
ROC_AUC	0.9815
MCC	0.8783
PRECISION	0.8176
RECALL	0.9937
PR_AUC	0.8402

CONFUSION (FULL SESSION)

True positives	64207
False positives	14325
True negatives	281289
False negatives	406
Accuracy	0.9591
FPR	0.0485

ATTACK CATEGORY COVERAGE

Normal	295614
Generic	55820
Exploits	3643
Fuzzers	3624
DoS	914
Reconnaissance	515
Analysis	77
Backdoors	10
Shellcode	9
Worms	1

ANALYST ACTIONS (HUMAN-IN-THE-LOOP)

Contained (analyst)	4577
Approved	60013
Overridden	14325
Total feedback events	78915

LIVE POLICY DRIFT

Drift vs training prior	0.0403

Fig. 9. Consolidated triage session report for the full-feed run, providing a self-contained, auditable record of model provenance, action tiers, detection performance, the full-session confusion matrix, attack-category coverage across all ten classes, human-in-the-loop analyst actions, and live policy drift for the complete 360,227-transition streaming session.

External validity. The study uses two feature-rich, identifier-complete datasets; whether the observed method ranking transfers to sparse-feature regimes (e.g. raw NetFlow) remains open and is left to future work. The two datasets nonetheless differ substantially in feature construction and observation dimension (46 Argus/Bro-derived features versus ≈75–77 CICFlowMeter statistics) and in attack composition, and the agent ranking holds across both, so the ranking is not an artifact of a single feature-extraction pipeline. On CIC-IDS-2017 the day-segregated attack schedule leaves only two folds satisfying our inclusion criterion, so its temporal-variance estimates rest on fewer folds than UNSW-NB15's; we accordingly treat the CIC-IDS-2017 temporal-sensitivity results as indicative rather than definitive, while noting that the agent ranking is nonetheless replicated across both evaluated folds. Timing metrics are reported in episode steps rather than wall-clock, and deployment validity beyond our advisory, human-in-the-loop setting would require a sustained shadow-mode track record on live telemetry; the advisory posture is itself the mitigation in the interim, since the deterministic severity floor and analyst-owned containment (Section 6) bound the consequences of policy error while such a record is accumulated. Finally, the near-unit ROC-AUC of the supervised baselines (Table 8) indicates that UNSW-NB15 is highly separable under the evaluated detection formulation and may reflect a small number of near-separating features; we did not perform a feature-level ablation to identify them, and doing so is left to future work. Temporal leakage is unlikely to explain this separability, since every partition is strictly chronological and verified by the post-split audit of Section 3.8. The separability is moreover a property of the binary detection task rather than of the triage policy: the offline agents remain well below the supervised baselines on MCC (0.61–0.75 versus ≈0.97; Tables 8 and 10) and policy recovery is only moderate (Section 5.4), so the benchmark is not saturated on the axes we report. Because ROC-AUC saturates under this separability, we treat MCC as the primary ranking metric throughout (Section 4.5), which is what exposes the degenerate value-based and Decision-Transformer policies that ROC-AUC conceals. Replication on CIC-IDS-2017 (Section 5.7) yields the same ranking at lower absolute scores.

Conclusion validity. All comparisons use multi-seed evaluation with non-parametric Friedman and Nemenyi tests, reducing the risk that reported differences reflect seed or distributional assumptions. Statistical power is nonetheless limited on CIC-IDS-2017 by its two-fold restriction and by five seeds per fold; we address this by reporting effect structure (mean ranks) and cross-fold replication alongside p-values rather than relying on significance thresholds alone. We also distinguish the evidence base of each table: Tables 8, 10, 11 and the policy-recovery analysis (Section 5.4) pool 5 folds × 5 seeds, whereas Tables 6, 7, and 9 are single-fold and labeled as such. Given how strongly single-fold results differed from the multi-fold findings for DT and CQL, the single-fold ablations should be read as indicative, and a multi-fold ablation is the natural confirmation.

8. Conclusion and future work

This work makes offline reinforcement learning practical for security operations center (SOC) alert triage on standard flow-based intrusion datasets, and subjects it to an evaluation protocol designed to withstand the leakage and optimism that inflate much of the intrusion-detection literature. Our first contribution, pseudo-trajectory reconstruction, converts flat flow tables into offline decision episodes through causal host grouping, temporal segmentation, and multi-objective reward assignment. Our second contribution is a strictly leakage-free temporal protocol — chronological partitioning, training-only normalization, expanding-window cross-validation, and a post-split audit — together with a uniform, model-independent fold-inclusion criterion that prevents policies trained on attack-scarce periods from being scored as if they had learned to triage.

Benchmarking seven offline policy-learning agents across two identifier-complete, feature-rich datasets under this protocol yields a consistent overall pattern. The reward-free recurrent imitator, Recurrent Behavioral Cloning, attains the highest detection MCC on both UNSW-NB15 (0.752) and CIC-IDS-2017 (0.621), while also showing low across-fold sensitivity. Comparison with the memoryless imitator and the advantage-weighted recurrent policy helps isolate the observed gains: recurrence and trajectory context appear to contribute more consistently than reward-derived advantage weighting, which does not reliably improve performance and increases seed variance. In contrast, value-based methods (CQL, IQL) and the Decision Transformer exhibit degenerate all-positive or all-negative detection behavior, which can be obscured by F1 and ROC-AUC but is revealed by the Matthews correlation coefficient. The recurrence of this behavior across both datasets, including CIC-IDS-2017, suggests that it is not specific to a single data partition. These results therefore support the use of multi-metric, stability-aware reporting when evaluating offline-RL methods for security triage.

Because these properties make the recurrent policies dependable yet inherently conservative — like the analysts they imitate, they escalate to containment only rarely — we position the system for advisory, human-in-the-loop deployment rather than autonomous response. For deployment we favor the lower-variance advantage-weighted GRU, composed with a deterministic severity floor so that the policy accelerates the high-volume Dismiss/Investigate decision while no medium- or high-severity alert is auto-dismissed and containment remains an analyst-owned action. Reflecting the paper's central finding, production monitoring is anchored on confusion-matrix–based signals (MCC) rather than threshold-dependent scores alone, so that silent degeneration of the kind we observe offline can be detected before it affects analyst workload.

Future work. Several directions follow. First, the causal grouping on host identifiers should be validated against real analyst-session logs, moving from a structural episode convenience to an operationally grounded one. Second, the reconstruction parameters — the temporal gap threshold τ and the episode-length bounds — warrant systematic sensitivity analysis. Although our framework supports such analysis, we leave this sweep to future work. Third, an advisory deployment should be validated in a live *shadow-mode* study over weeks to months of production traffic, measuring analyst agreement, the false-dismiss rate on medium/high-severity alerts, the review-volume overhead of the severity floor, and stability under concept drift with periodic retraining. Fourth, the method ranking should be tested in sparse-feature regimes such as raw NetFlow. Finally, the single-step triage policy can be evolved into an agentic, multi-step SOC analyst that plans investigative actions rather than emitting a single tier per flow while preserving accountability through confidence-based safety gating, bounded human oversight, and continuous drift monitoring. A promising direction is to extend the Leakage-Free Benchmark of Offline Policy Learning for SOC Alert Triage to dynamic graph decision-making, advancing prior work on multilingual entity alignment and infectious-disease network dismantling (Akhtar et al., 2025, 2026). This could move beyond static prediction or node identification toward leakage-free offline RL for sequential, cost-aware interventions and evidence selection across evolving knowledge and social networks, establishing a broader framework for reliable offline decision-making on dynamic graphs.

Funding

This research was funded by Naif Arab University for Security Sciences, Saudi Arabia under grant No. NAUSS-26-R03.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author acknowledges using Grammarly for language and grammar refinement. All revisions suggested by the tool were carefully reviewed and validated by the author, who takes full responsibility for the accuracy, originality, and integrity of the manuscript.

Appendix. Implementation details and hyperparameters

This appendix reports the complete configuration used to produce the results in Section 5. To isolate methodological effects, all architecture, optimization, reconstruction, and reward settings are held constant across both datasets. The only intentional cross-dataset difference is the training budget, which is larger for CIC-IDS-2017 because of its higher feature dimensionality and stronger day-segregated distribution shift. All agents are trained and evaluated using five random seeds $\{42, 123, 456, 789, 999\}$ under expanding-window temporal cross-validation. Table 17 reports the settings shared by all agents and both datasets; Table 18 the dataset-specific configuration; Table 19 the imbalance-aware action-loss settings; Table 20 the algorithm-specific hyperparameters; and Table 21 the multi-objective reward specification.

Data availability

Data will be made available on request.

Table 17
Shared training hyperparameters used across all agents and datasets.

Hyperparameter	Value	Notes
Action space	3 (tier)	Dismiss/Investigate/Contain
Hidden width	64	Per layer
Number of layers	2	–
Dropout	0.1	–
Optimizer	Adam	–
Learning rate	3×10^{-4}	–
Weight decay	1×10^{-5}	–
Batch size	256	–
Gradient clip	1.0	Global norm
Discount factor γ	0.9	–
Evaluation interval	1000	Steps
Random seeds	$\{42, 123, 456, 789, 999\}$	

Table 18
Dataset-specific configuration. Architecture, optimization, reward, and reconstruction settings are identical across datasets; only the training budget differs.

Setting	UNSW-NB15	CIC-IDS-2017
Total flows	2,540,047	3,119,345 raw/2,830,743 used
Attack prevalence	≈12.6%	≈19.7%
Attack categories	9 + normal	8 + normal (day-seg.)
Observation dimension	46	≈75–77 (CICFlowMeter)
Training steps (per agent)	6000	6000
Temporal design	5-fold exp. window	5-fold exp. window
Folds evaluated	5 of 5	2 of 5 (folds 4–5)
Fold inclusion	Train-window attack rate $\geq 1.5\%$	
Feature scaler	RobustScaler (median/IQR), train only	
Grouping key	(srcip, dstip), causal host pairs	
Minimum group size	2 (smaller groups discarded)	
Temporal gap bound τ	60 s episode-split threshold	
Max/min episode length	50/2	

Table 19
Imbalance-aware action-loss configuration for the sequence and recurrent agents.

Hyperparameter	Value	Notes
Class-weighted CE	Enabled	Inverse-frequency action weighting
Focal focusing γ_f	1.5	$\gamma_f = 0$ recovers standard CE

Table 20
Algorithm-specific hyperparameters shared across datasets. The recurrent GRU/LSTM policies use the advantage-weighted objective of Eq. (5).

Agent/family	Hyperparameter	Value
Decision transformer	Context length K	25
	Embedding dimension	128
	Attention heads	4
	Transformer layers	3
GRU/LSTM (recurrent)	Context length	20
	Objective	Adv.-weighted
CQL	Conservative weight α	5.0
IQL	Expectile τ	0.7
	Inverse-temperature β	3.0
	Target update frequency	100 steps
BC	Supervised imitation (no extra hyperparameters)	

Table 21
Multi-objective reward specification used for all datasets and agents.

Outcome	Action tier	Reward
True attack	Contain	+1.5
True attack	Investigate	+0.5
True attack	Dismiss	−1.0
Benign	Dismiss (ignore)	+0.2
Benign	Dismiss (log)	+0.1
Benign	Investigate	−0.3
Benign	Contain	−0.6

References

Akhtar, M. U., Liu, J., Xie, Z., Cui, X., Liu, X., & Huang, B. (2025). Multilingual entity alignment by abductive knowledge reasoning on multiple knowledge graphs. *Engineering Applications of Artificial Intelligence*, *139*, Article 109660. http://dx.doi.org/10.1016/j.engappai.2024.109660.

Akhtar, M. U., Wajahat, A., Razzaq, A., Ahmed, I., Farman, H., Khan, K., & Khan, D. (2026). Computational modeling and analysis of behavioral patterns for infectious disease networks dismantling. In *Computational and deep learning models for advanced behavioral analysis* (pp. 365–390). IGI Global Scientific Publishing, http://dx.doi.org/10.4018/979-8-3373-5062-2.ch011.

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX security symposium* (pp. 3971–3988).

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., & Mordatch, I. (2021). Decision transformer: Reinforcement learning via sequence modeling. *Advances in Neural Information Processing Systems*, *34*, 15084–15097.

Gueriani, A., Kheddar, H., & Mazari, A. C. (2023). Deep reinforcement learning for intrusion detection in IoT: A survey. In *2023 2nd international conference on electronics, energy and measurement* (pp. 1–7). IEEE.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Huiying, D. (2026). Research on optimizing network intrusion detection using deep learning and big data in intelligent elderly care. *Computer Standards & Interfaces*, Article 104136.

Jamshidi, S., Nikanjam, A., Nafi, K. W., Khomh, F., & Rasta, R. (2025). Application of deep reinforcement learning for intrusion detection in Internet of Things: A systematic review. *Internet of Things*, *31*, Article 101531.

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, *4*(9).

Kostrikov, I., Nair, A., & Levine, S. (2021). Offline reinforcement learning with implicit q-learning. arXiv preprint arXiv:2110.06169.

Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, *33*, 1179–1191.

Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline reinforcement learning: Tutorial, review, and perspectives on open problems. arXiv preprint arXiv:2005.01643.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision* (pp. 2980–2988).

Louati, F., Ktata, F. B., & Amous, I. (2024). Enhancing intrusion detection systems with reinforcement learning: A comprehensive survey of rl-based approaches and techniques. *SN Computer Science*, *5*(6), 665.

Moustafa, N., Creech, G., & Slay, J. (2017). Big data analytics for intrusion detection system: Statistical decision-making using finite dirichlet mixture models. In *Data analytics and decision support for cybersecurity: Trends, methodologies and applications* (pp. 127–156). Springer.

Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference* (pp. 1–6). Ieee.

Mpoporo, L. J., Owolawi, P. A., & Tu, C. (2026). Deep reinforcement learning algorithms for intrusion detection: A bibliometric analysis and systematic review. *Applied Sciences*, *16*(2), 1048.

Mukil, S., Patel, N., Lamkuche, H. S., Alazaidah, R., & Taqatqa, S. (2026). Intrusion detection system using machine learning models. In *Business resilience and business innovation for sustainability: The double-edged role of artificial intelligence and other disruptive technologies* (pp. 2125–2139). Springer.

Neto, E. C. P., Iqbal, S., Buffett, S., Sultana, M., & Taylor, A. (2025). Deep learning for intrusion detection in emerging technologies: a comprehensive survey and new perspectives. *Artificial Intelligence Review*, *58*(11), 340.

Okafor, E., & Lee, S. (2026). Unsupervised learning framework for cyber threat detection, anomaly identification, and alert prioritization. *Applied Sciences*, *16*(4), 1884.

Pomerleau, D. A. (1988). Alvinn: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems*, *1*.

Ross, S., Gordon, G., & Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics* (pp. 627–635). JMLR Workshop and Conference Proceedings.

Yang, W., Acuto, A., Zhou, Y., & Wojtczak, D. (2026). A survey for deep reinforcement learning based network intrusion detection. *Applied AI Letters*, *7*(2), Article e70026.